THE UNIVERSITY OF ALABAMA®

WHERE LEGENDS ARE MADE®

Huffman Codes

Reducing the Length of a Message's Binary Encoding

CS 201: Data Structures and Algorithms
Jiacheng Li, Ph.D.

Huffman Codes

- Huffman codes are variable-length binary codes assigned to symbols based on their frequencies.
- Symbols that occur more often are given shorter codes, while less frequent symbols are given longer codes, resulting in efficient and lossless data compression.
- These codes are generated using the Huffman coding algorithm, which builds a binary tree that minimizes the total number of bits required to represent the data.

Set the Stage

- Alphabet A[1…n] with n characters
- Relative frequencies F[1…n] of each character
- Example

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1

Set the Stage (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1

- Assign binary codes C[1…n] to each character
  - Fixed-length: every code is the same length
  - Variable-length: codes can be different lengths
- Encode
  - Replace each character A[k] by its corresponding binary code C[k]
- Decode
  - Replace each binary code C[k] by its corresponding character A[k]

Goal

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	?	?	?	?	?	?	?	?	?	?

- Goal: to obtain shortest encoded message length
- Should we use fixed- or variable-length codes?

Goal

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	?	?	?	?	?	?	?	?	?	?

- Goal: to obtain shortest encoded message length
- Use variable-length binary codes
  - Larger frequency $\Rightarrow$ shorter binary code
  - Smaller frequency $\Rightarrow$ longer binary code
- To decode efficiently, the codes must satisfy the <u>Prefix property</u>
  - No character's code C[k] is a prefix of any other character's code C[k']
  - Without prefix property, which character is decoded?

Goal (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	?	?	?	?	?	?	?	?	?	?

- Goal: to obtain shortest encoded message length
- By mapping the characters to <u>variable-length</u> binary codes C[1…n] that satisfy the <u>prefix property</u> and yield the shortest encoded message lengths

Greedy Algorithm

- <u>Greedy Algorithm</u>
  - Makes the *optimal* available choice at each step.
  - Does not make decisions by looking at the bigger picture.
    - It's **greedy**
  - Does not always make the optimal answer.

Greedy Huffman Algorithm

1. Construct a Huffman tree (binary tree)
2. Assign each character a binary code

Greedy Huffman Algorithm (cont.)

1. Construct a Huffman tree (binary tree)

```
H = new Heap();    // Min-heap
for j = 1 to n
  H.insert( new Leaf(F[j], A[j]) ); // F[j] is key
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```

Greedy Huffman Algorithm (cont.)

```
H = new Heap();                    // Create an empty min-heap.
for j = 1 to n
  H.insert( new Leaf(F[j], A[j]) ); // F[j] is key
```

9	2	5	6	12	3	4	7	8	1
a	b	c	d	e	f	g	h	i	j

Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```

9	2	5	6	12	3	4	7	8	1
a	b	c	d	e	f	g	h	i	j

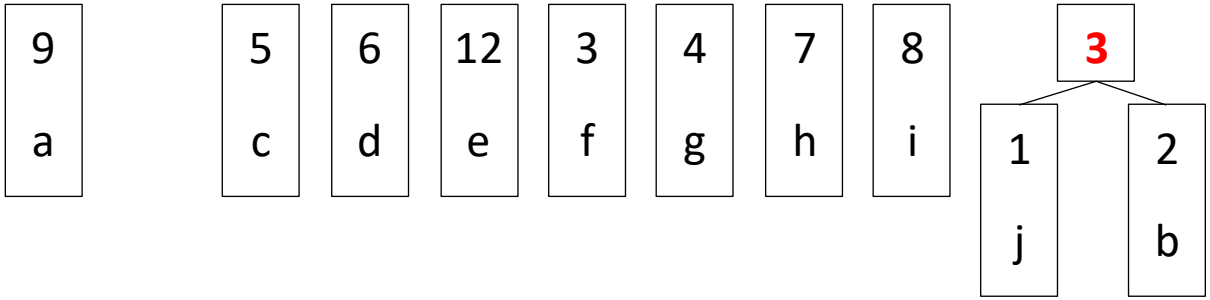
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```

9	**2**	5	6	12	3	4	7	8	**1**
a	**b**	c	d	e	f	g	h	i	**j**

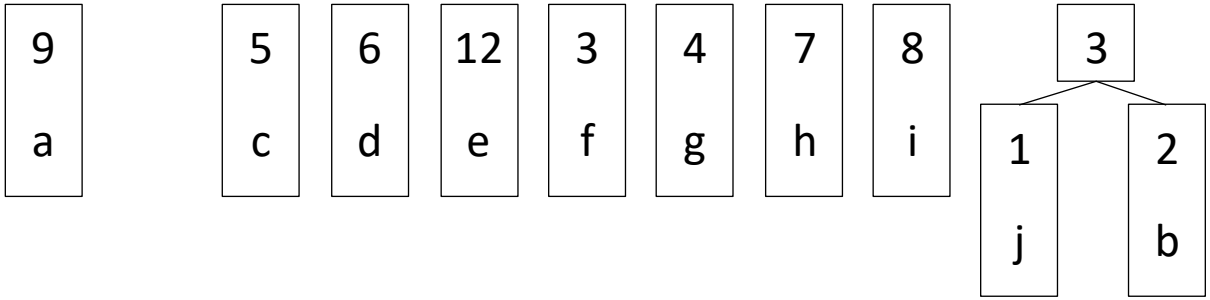
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



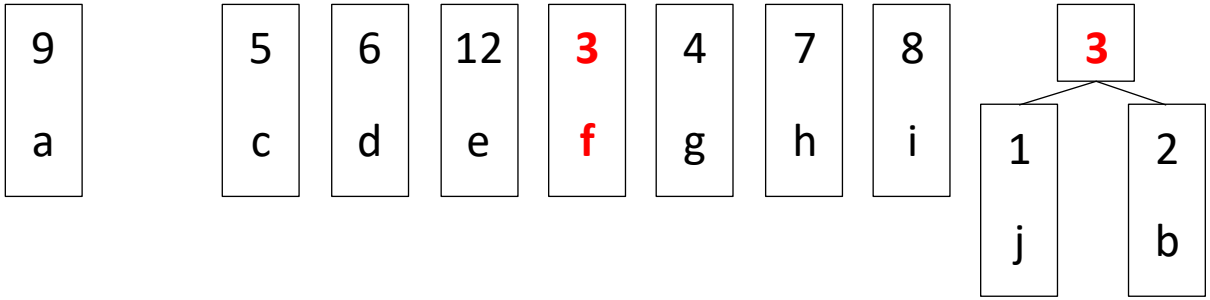
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



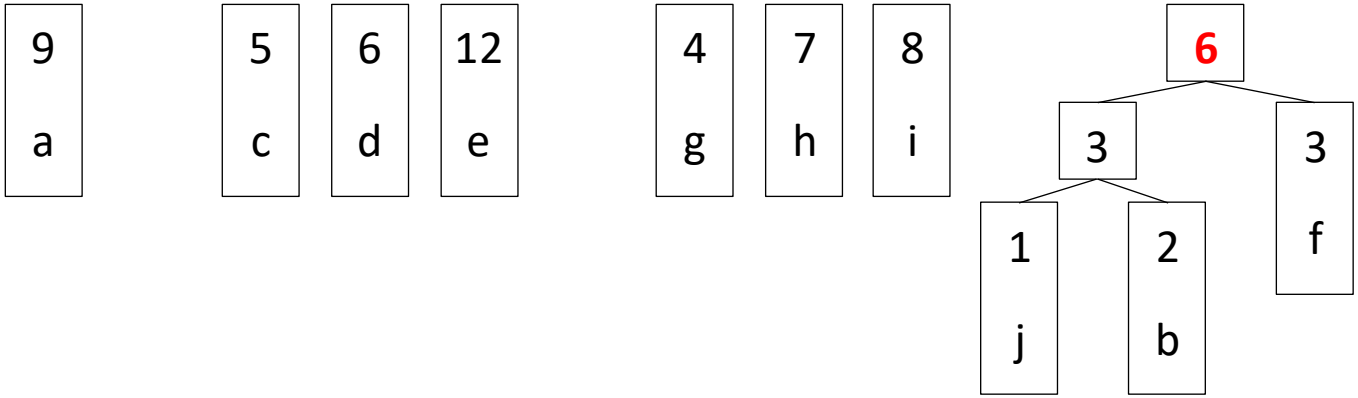
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



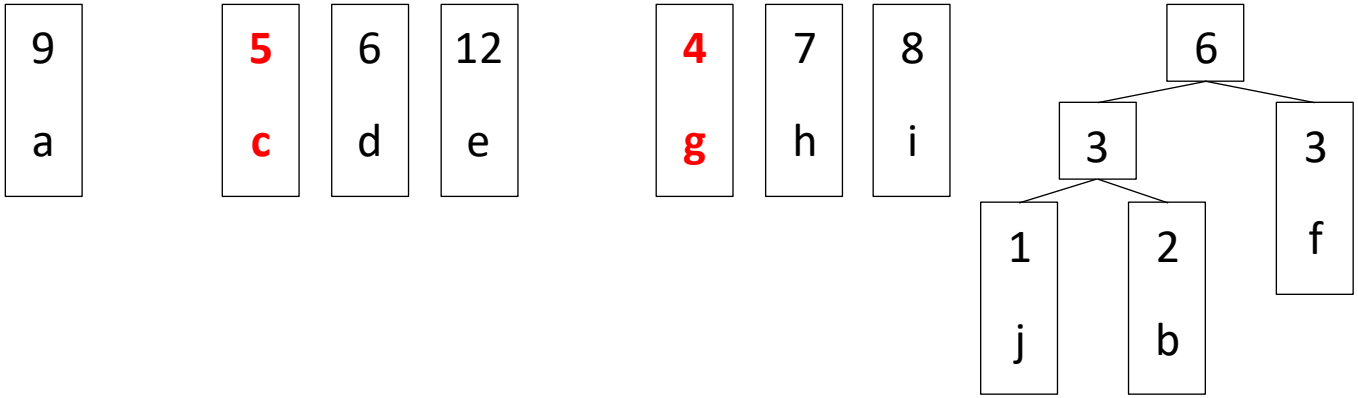
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



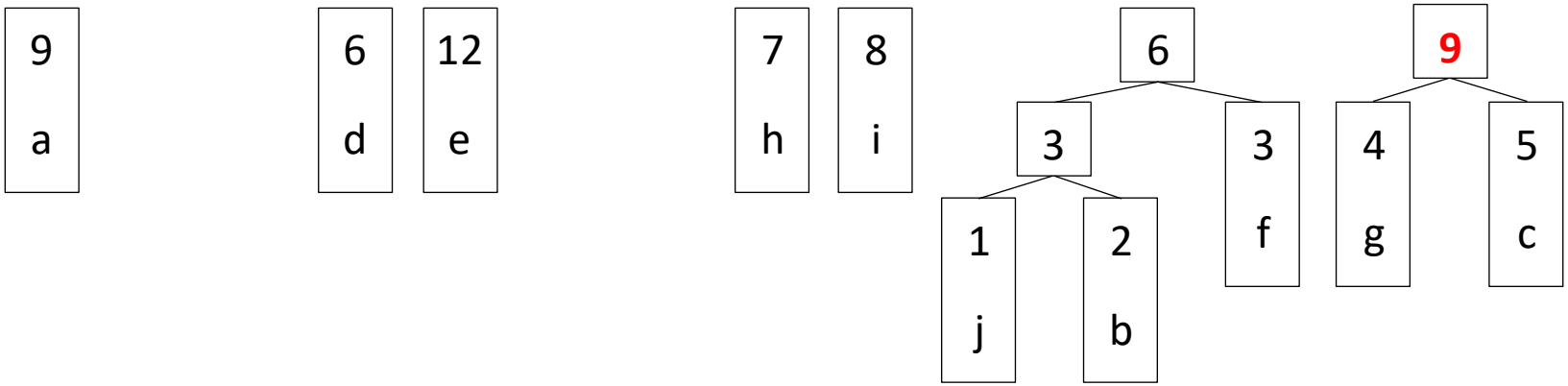
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



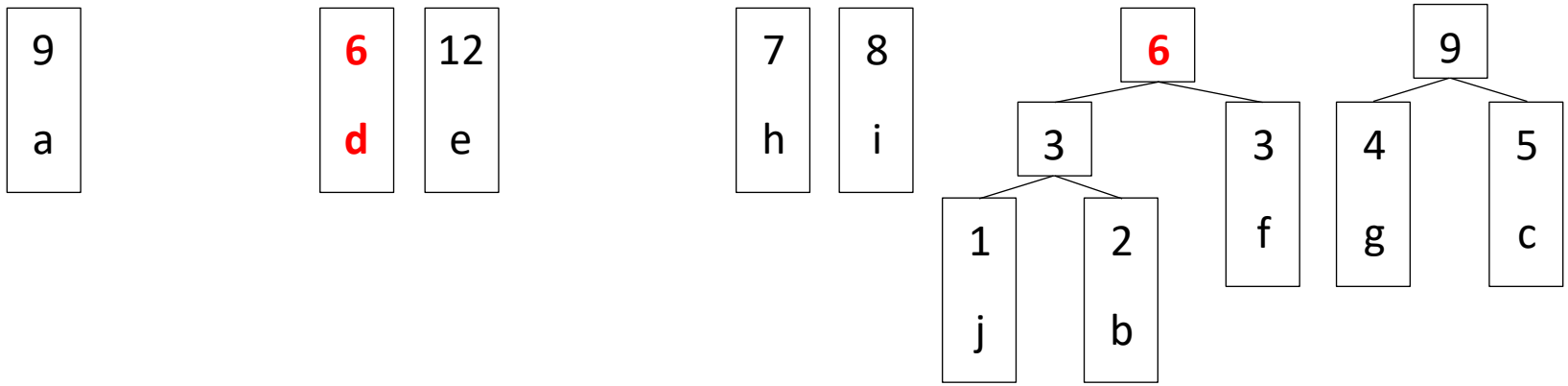
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



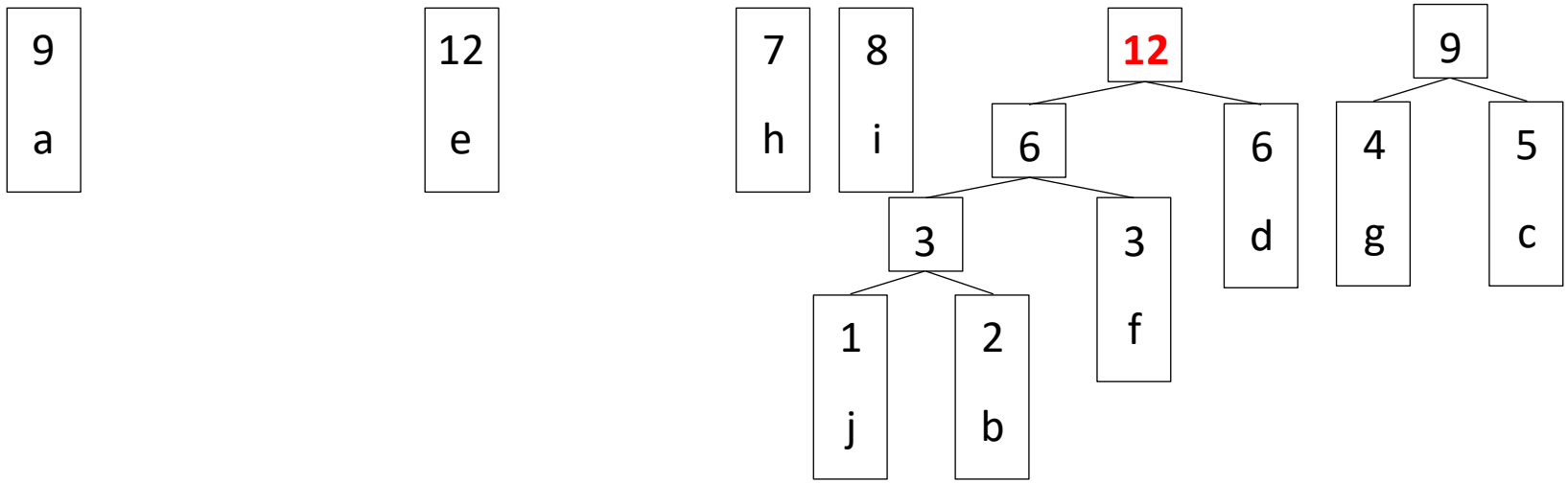
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



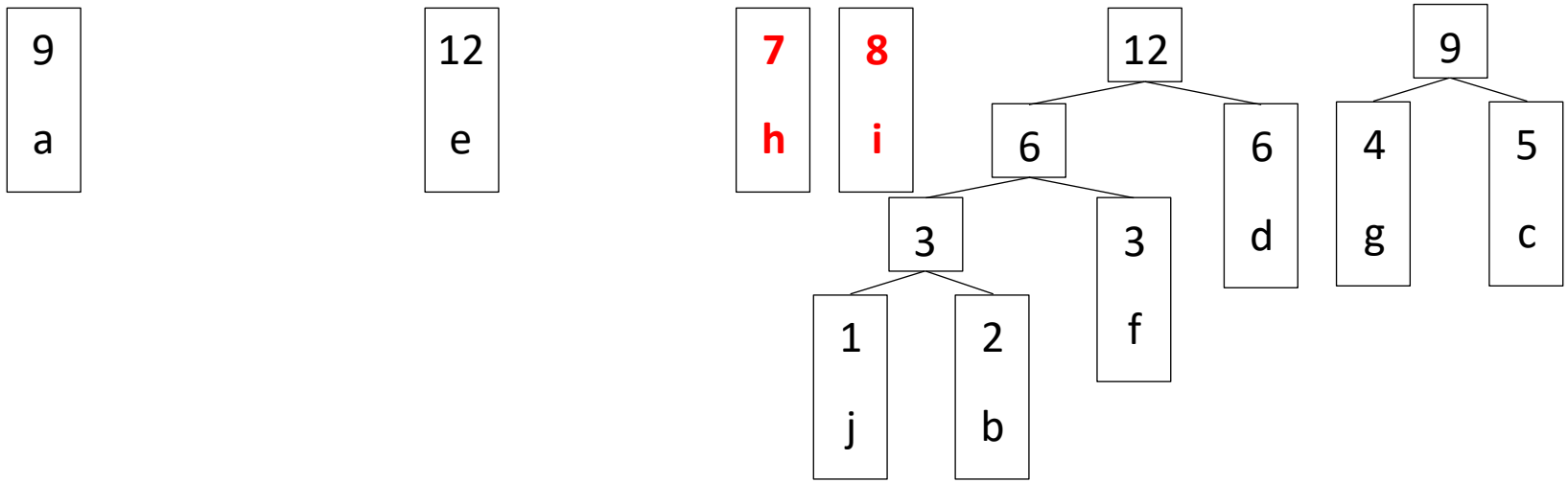
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



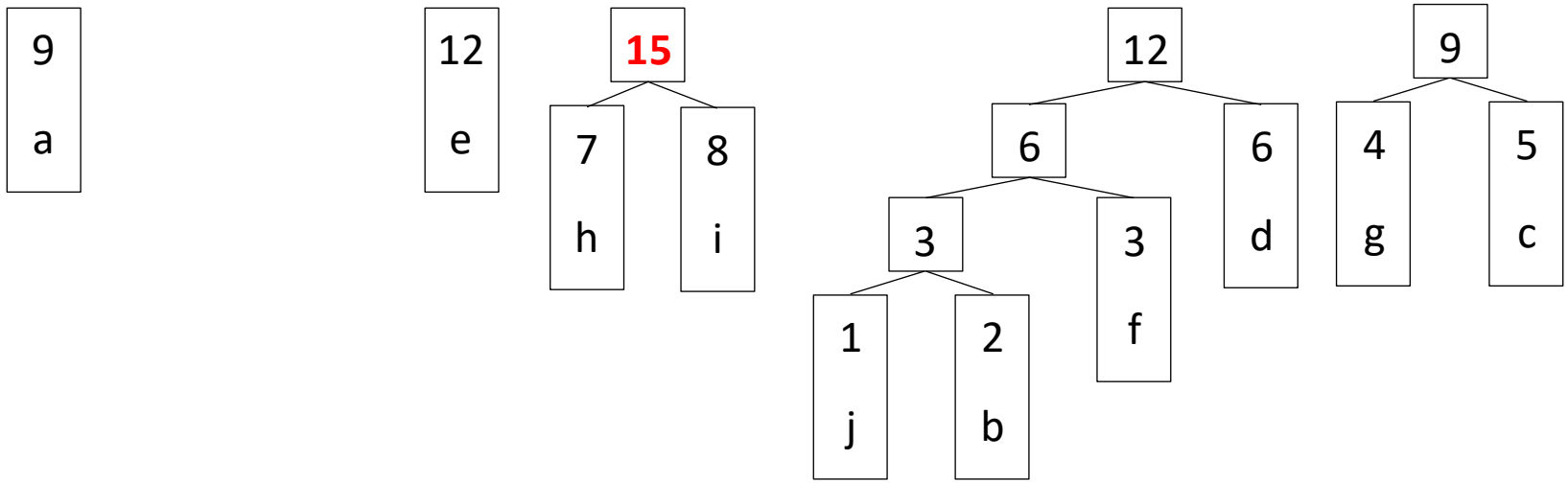
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



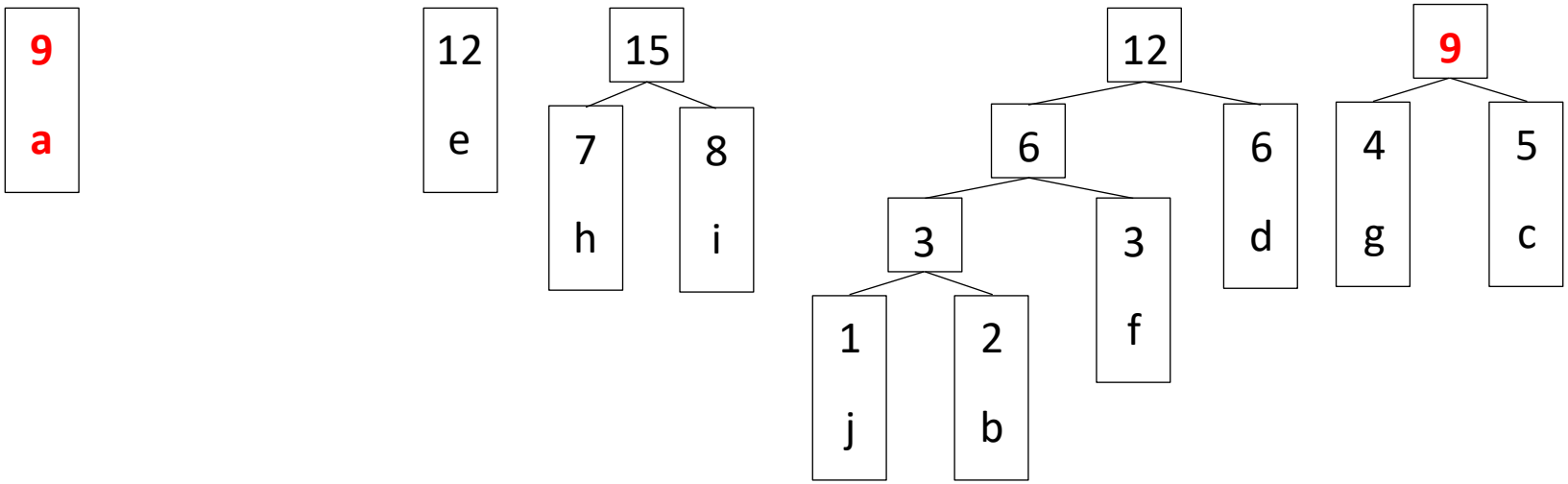
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



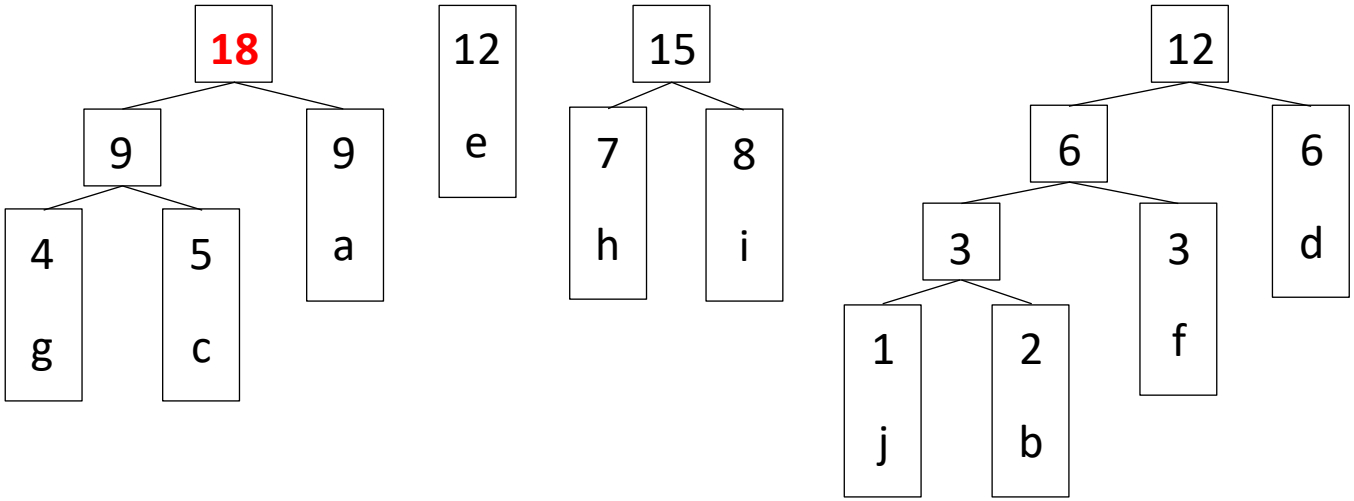
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



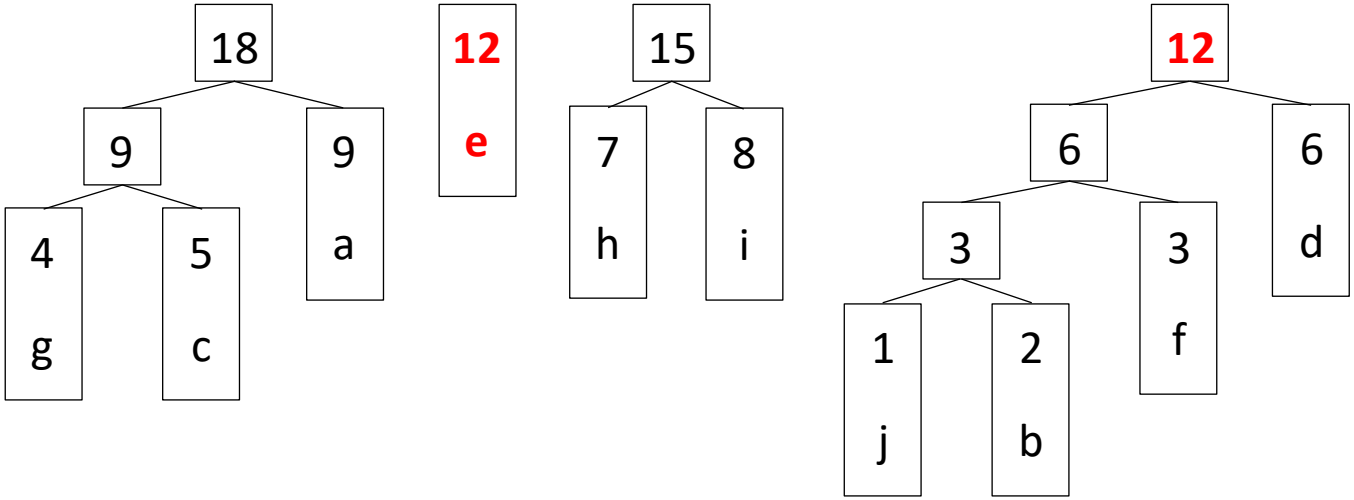
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



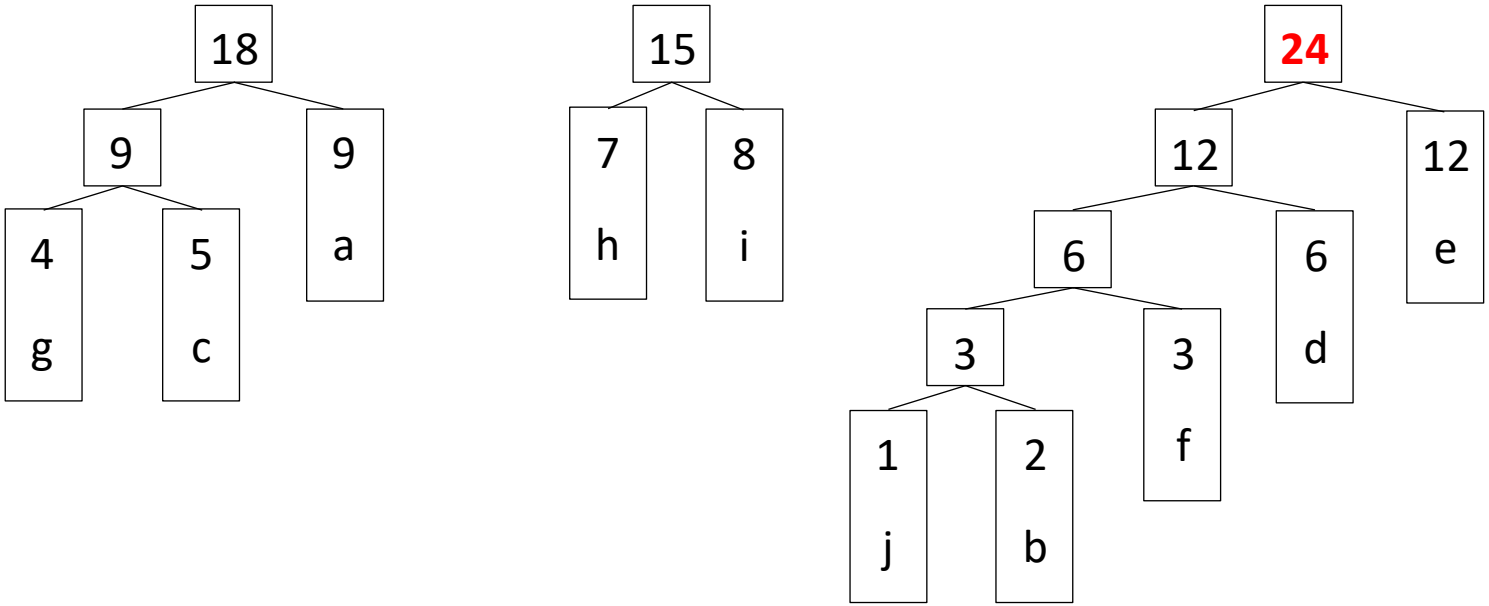
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



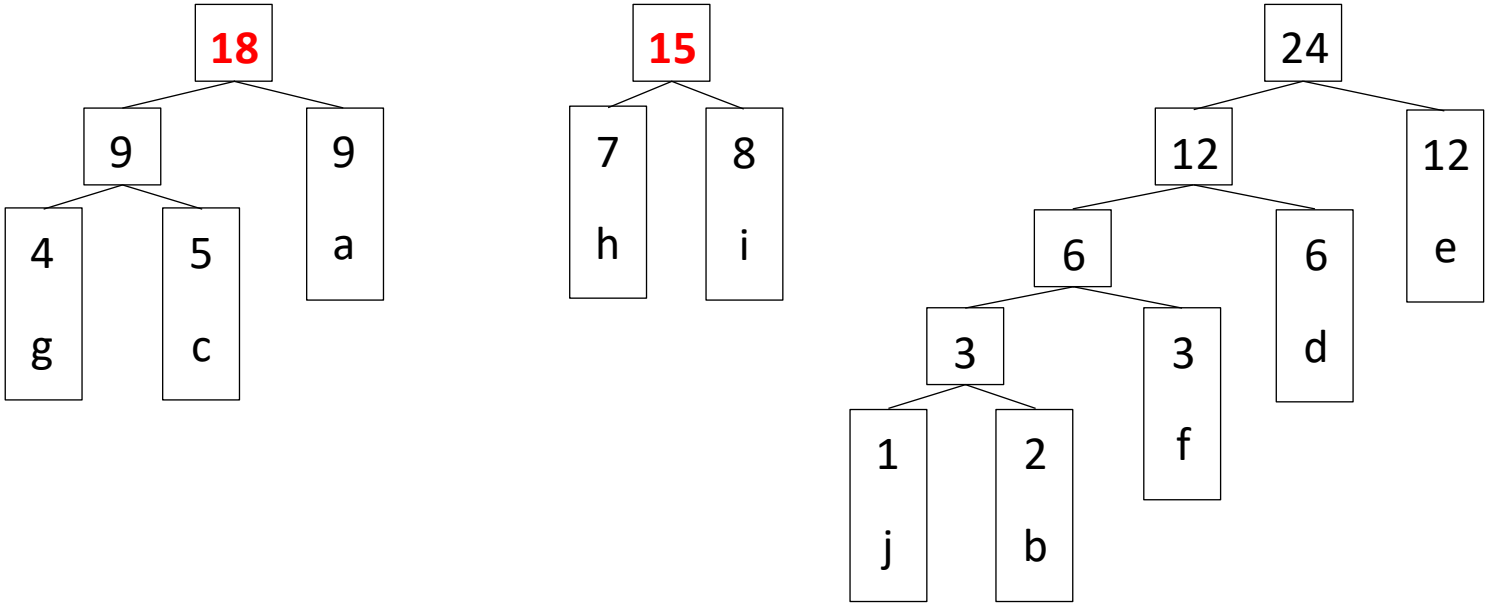
Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



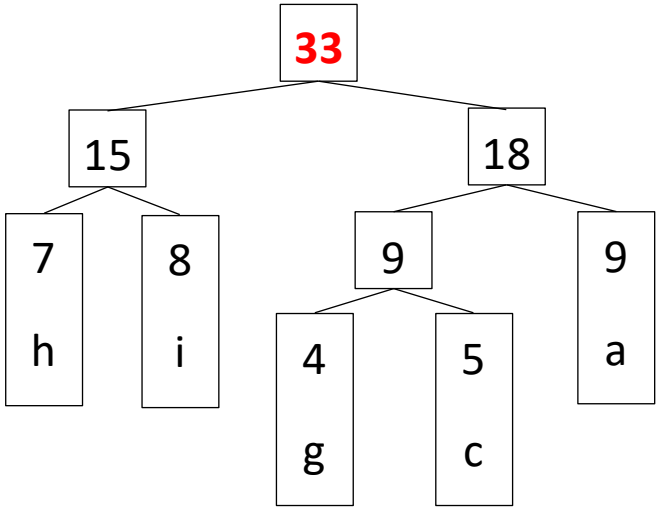
Greedy Huffman Algorithm (cont.)

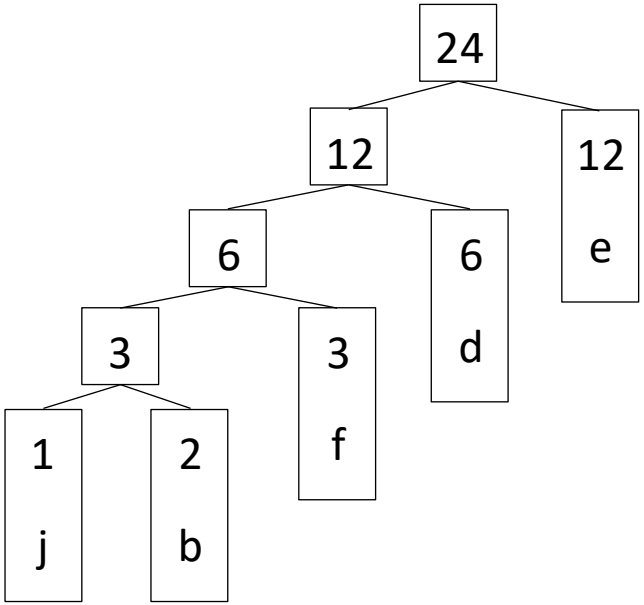
```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



Greedy Huffman Algorithm (cont.)

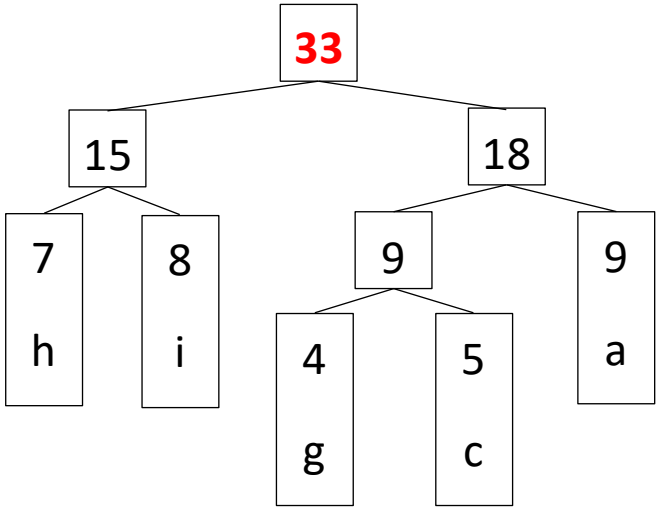
```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```

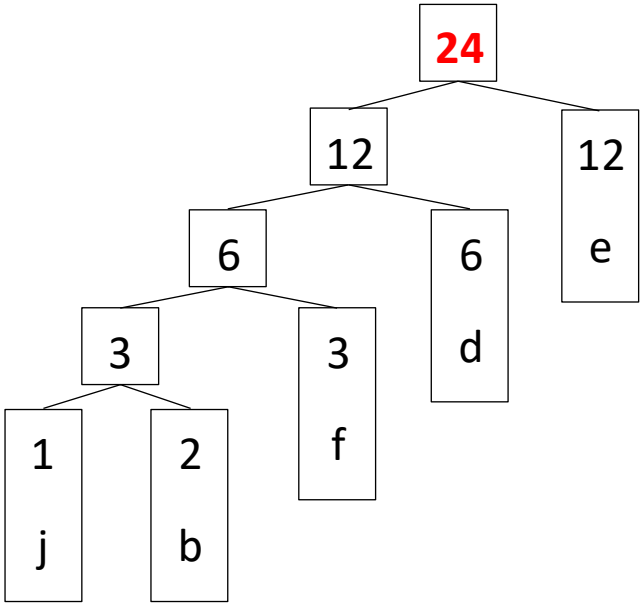




Greedy Huffman Algorithm (cont.)

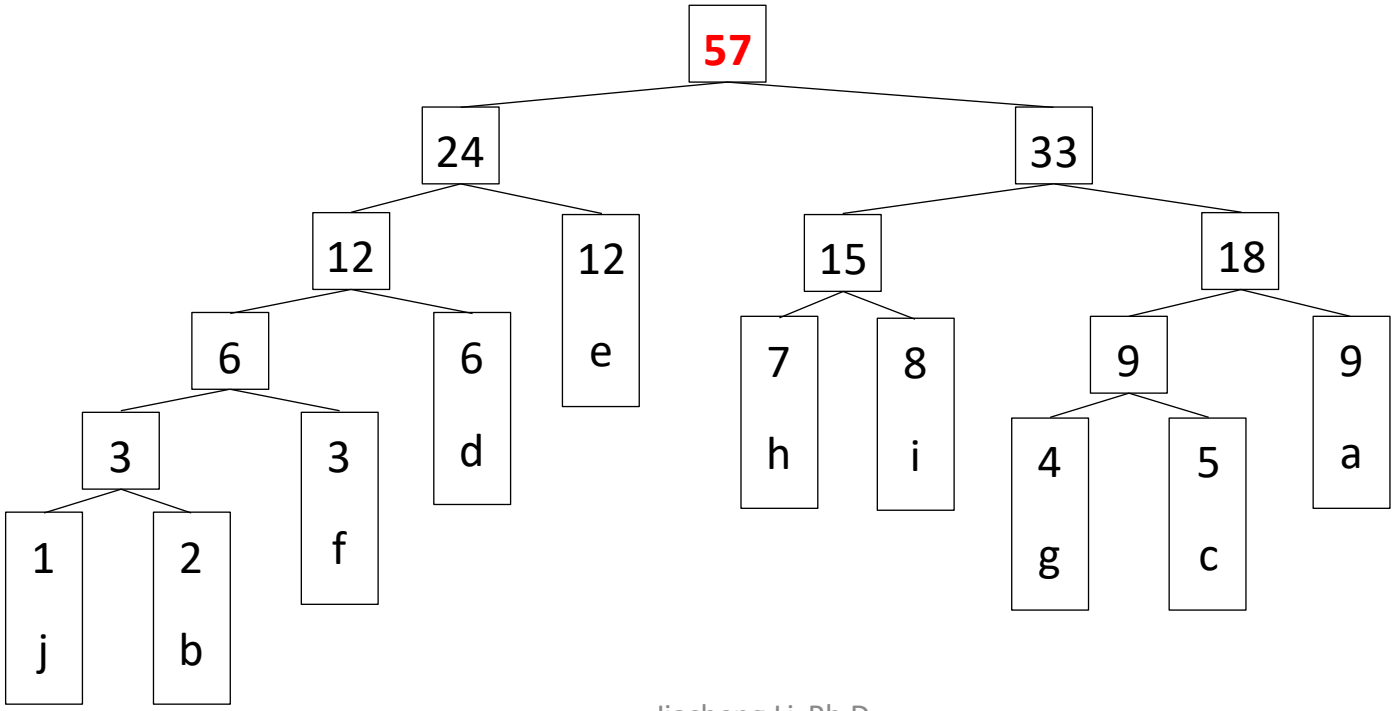
```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



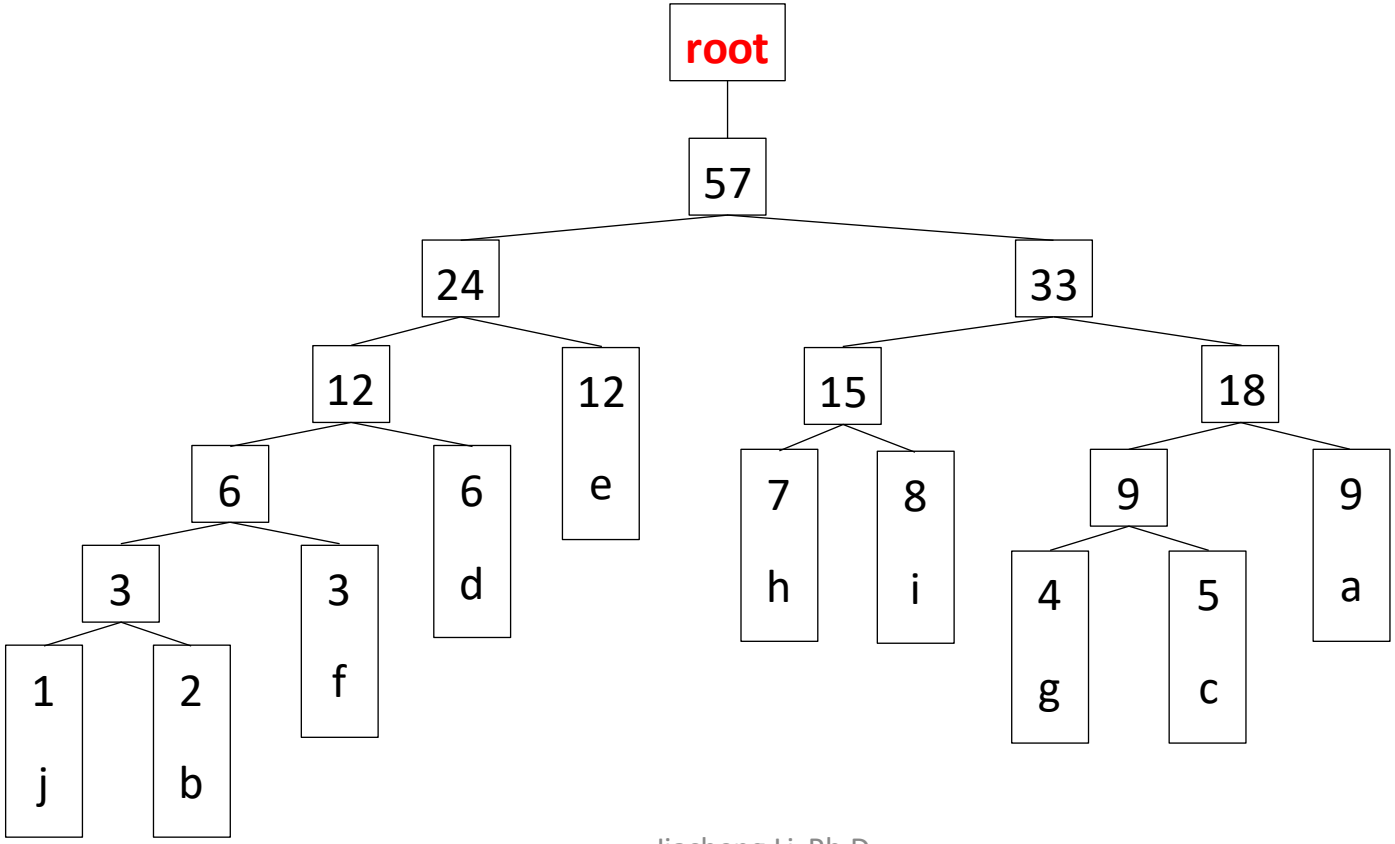


Greedy Huffman Algorithm (cont.)

```
while (H.size() >= 2) {
  left = H.removeMin();
  right = H.removeMin();
  sum = left.key + right.key;
  H.insert(new Node(sum, left, right)); // sum is key
}
```



Greedy Huffman Algorithm (cont.)



Greedy Huffman Algorithm (cont.)

2. Assign each character a binary code

Label each link to left child with 0

Label each link to right child with 1

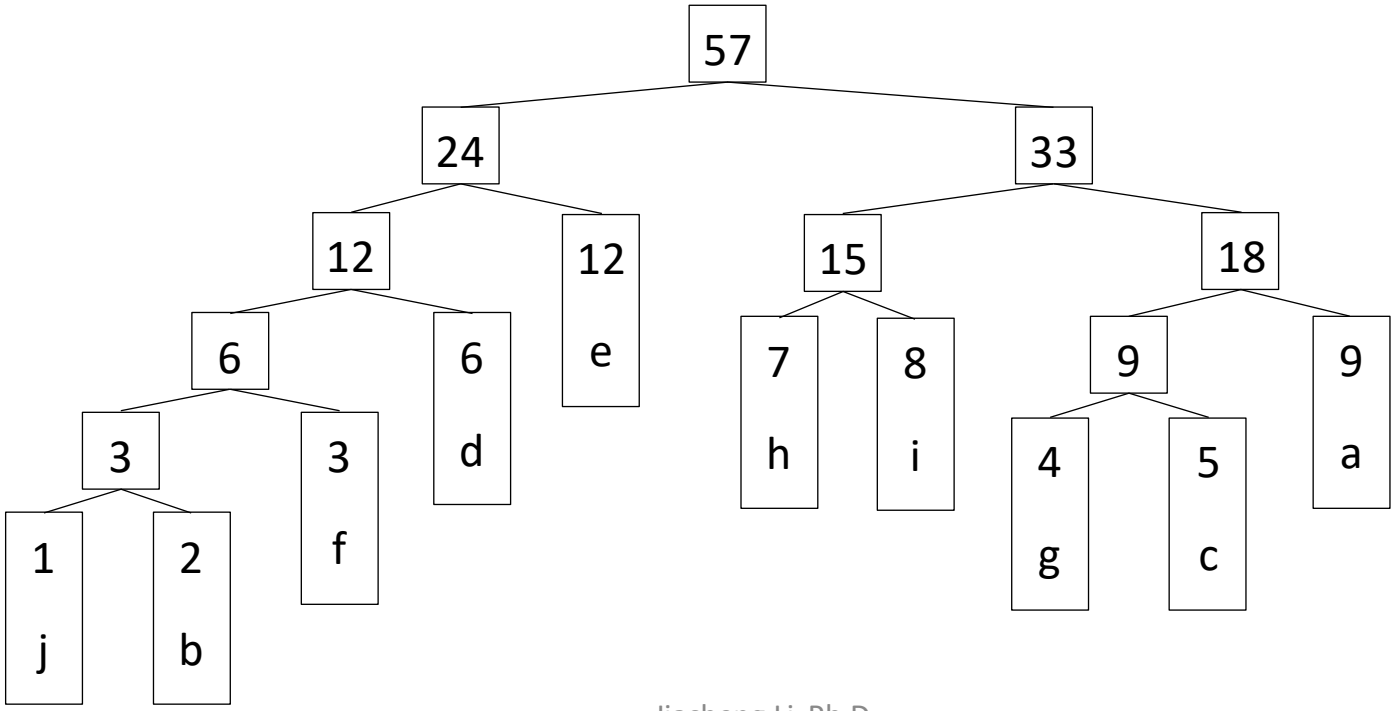
Follow the path from root to each leaf node

Concatenate the labels to obtain the binary code

Greedy Huffman Algorithm (cont.)

Label each link to left child with 0

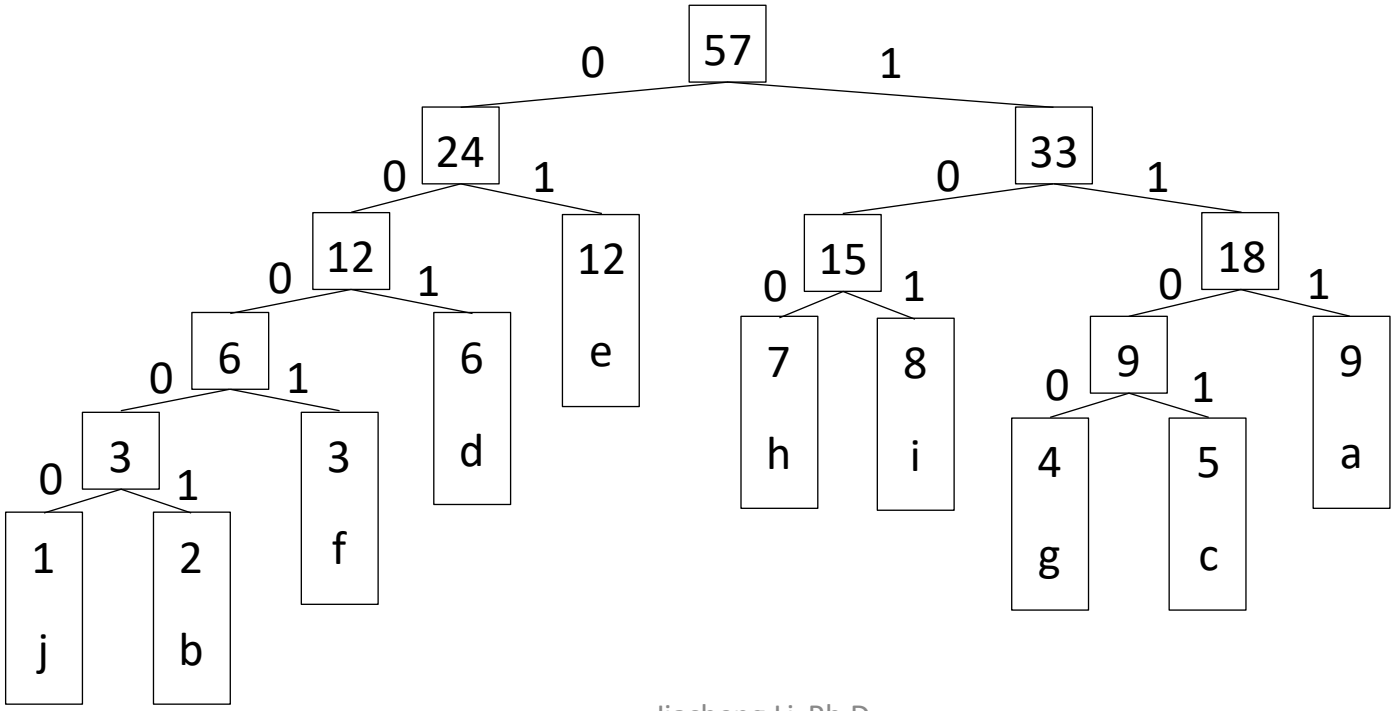
Label each link to right child with 1



Greedy Huffman Algorithm (cont.)

Label each link to left child with 0

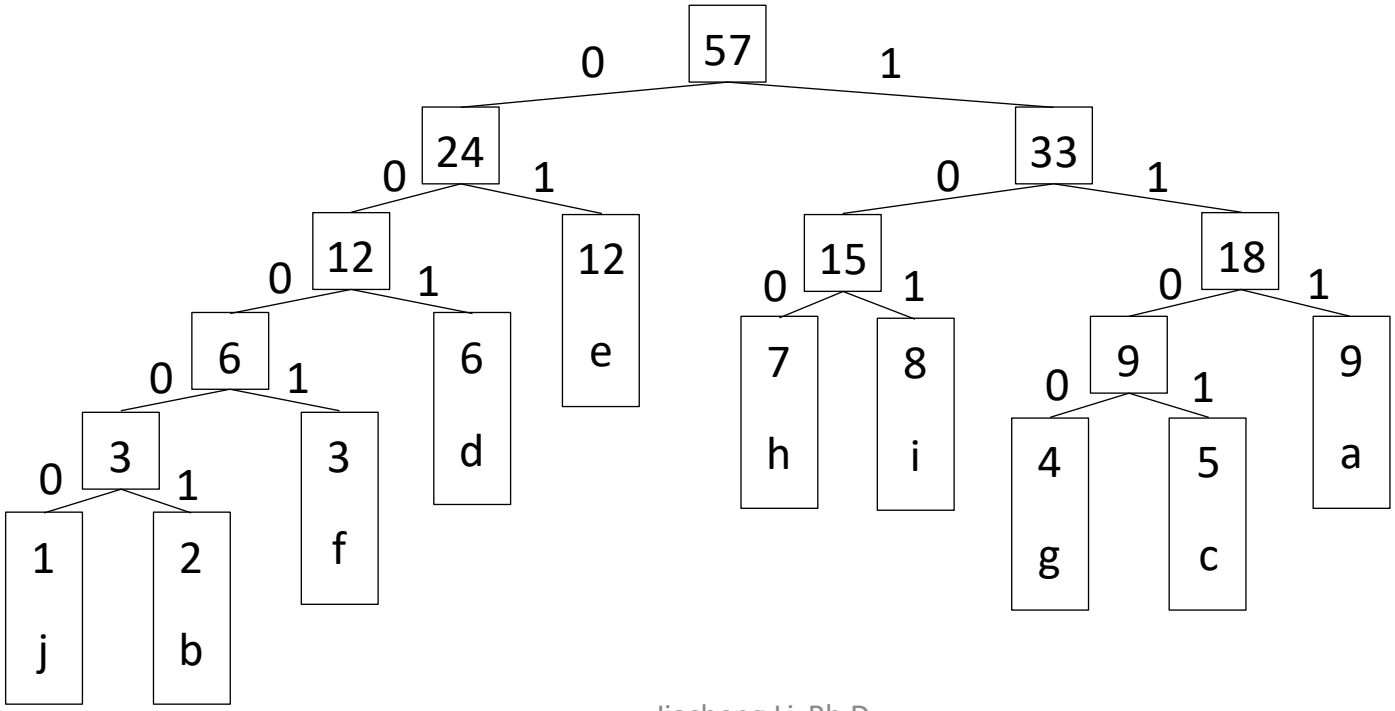
Label each link to right child with 1



Greedy Huffman Algorithm (cont.)

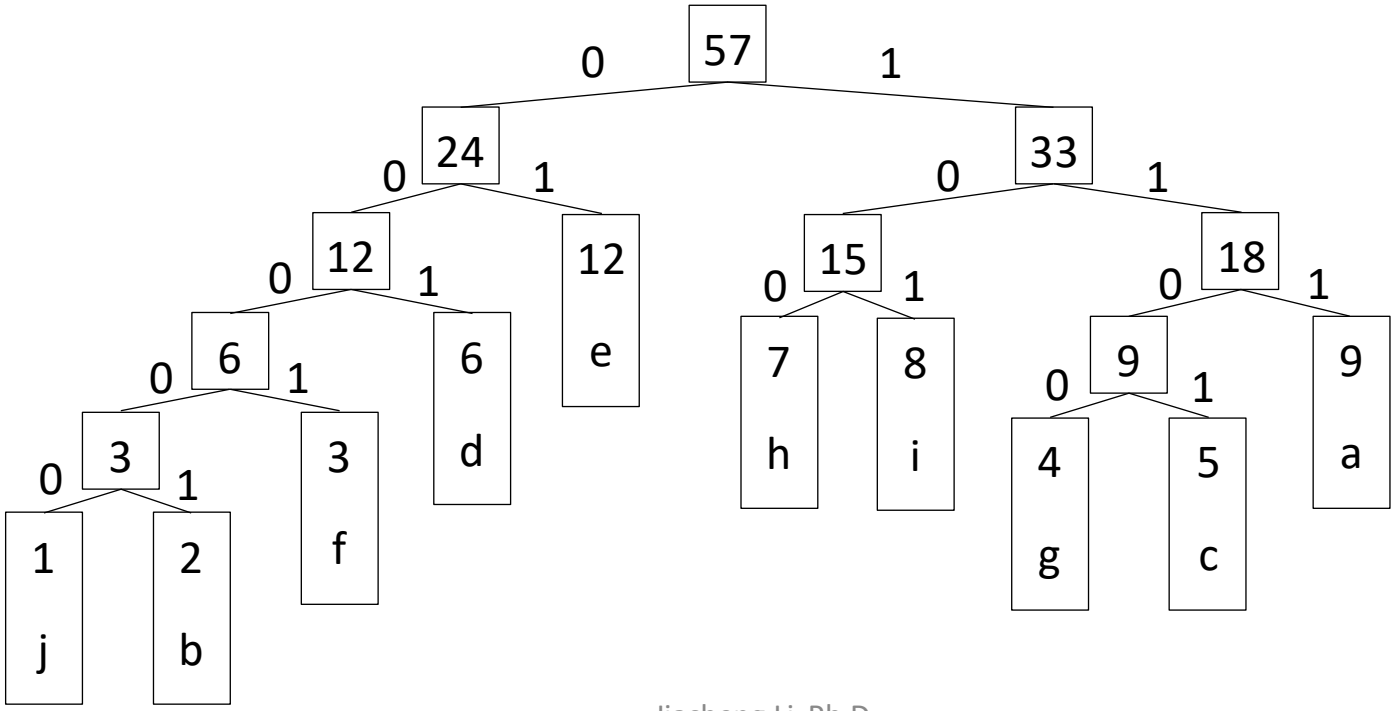
Follow the path from root to each leaf node

Concatenate the labels to obtain the binary code



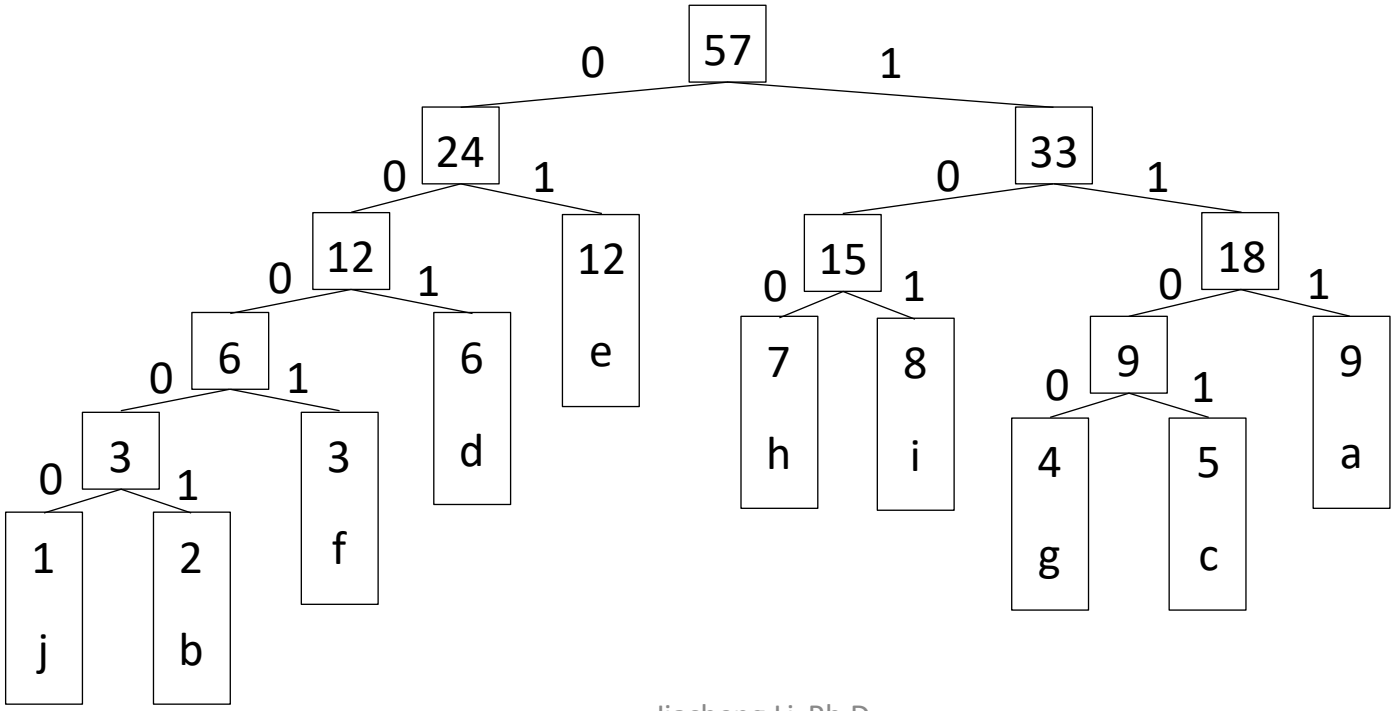
Greedy Huffman Algorithm (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	?									



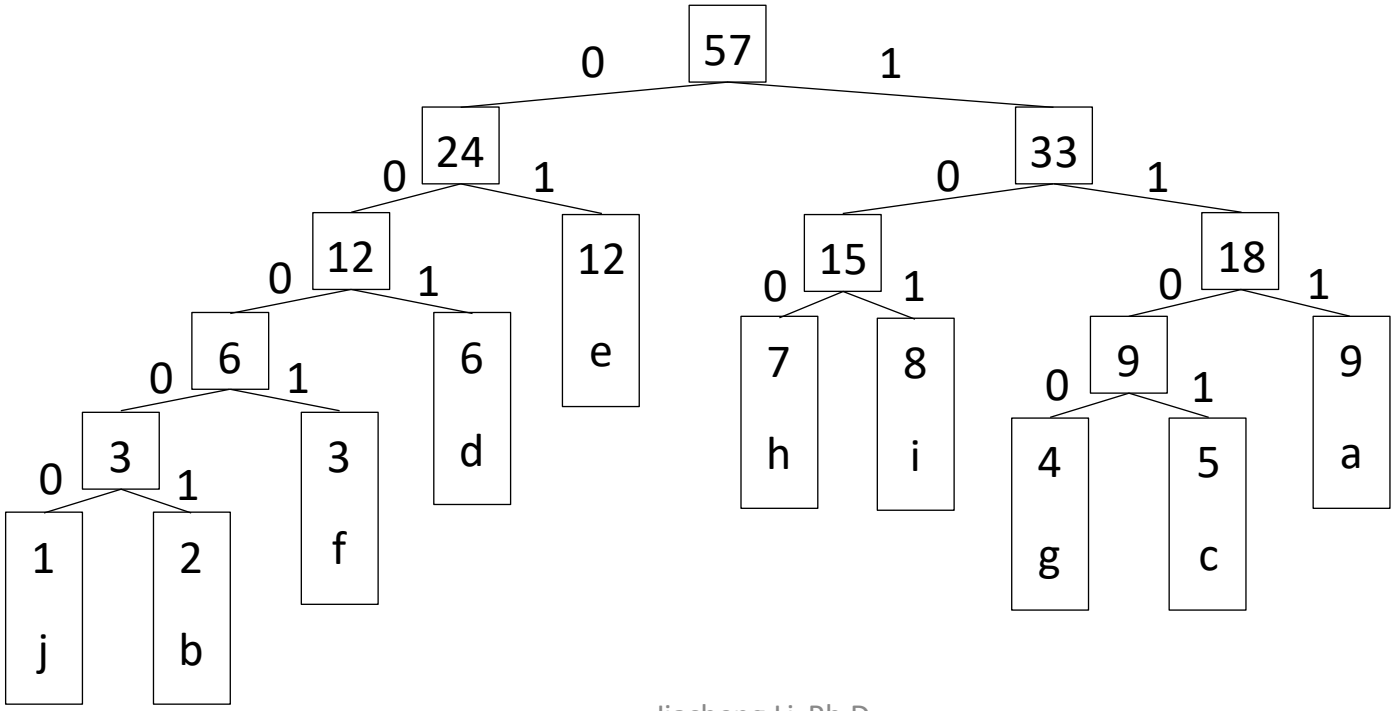
Greedy Huffman Algorithm (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	?								



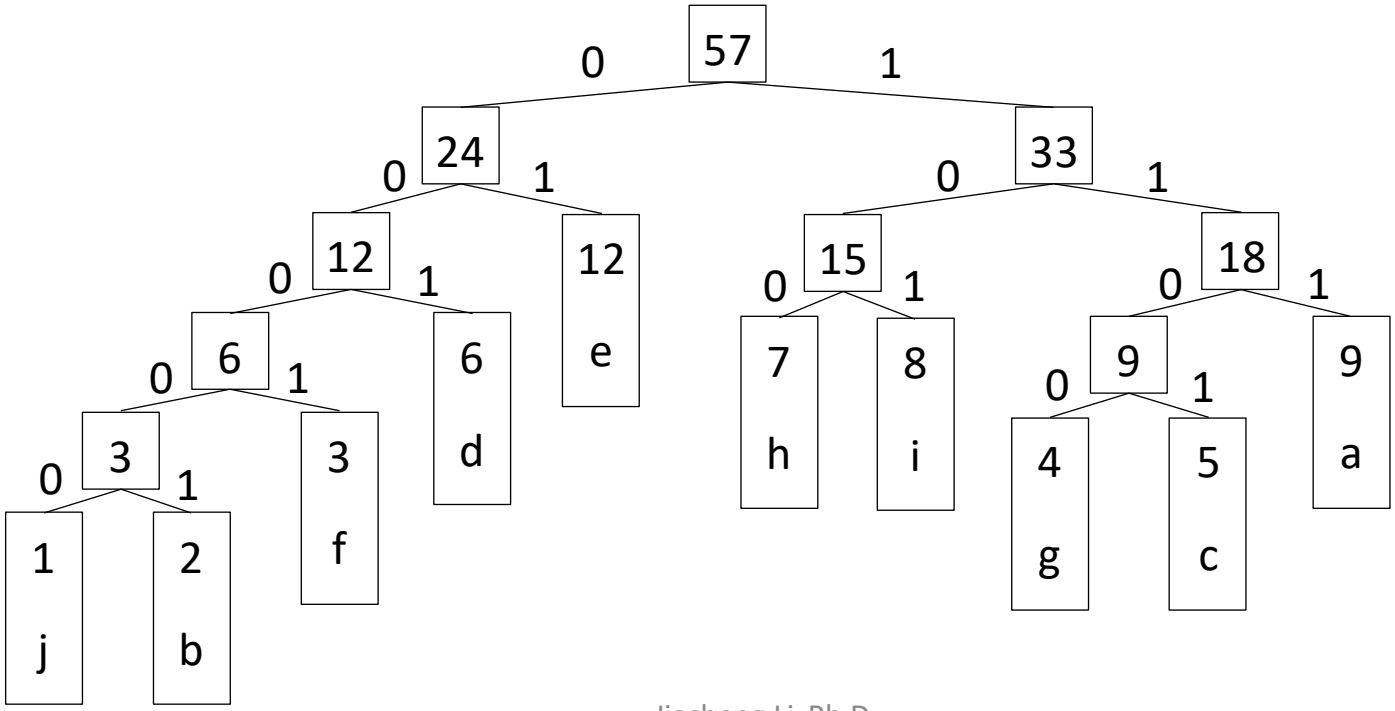
Greedy Huffman Algorithm (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	?							



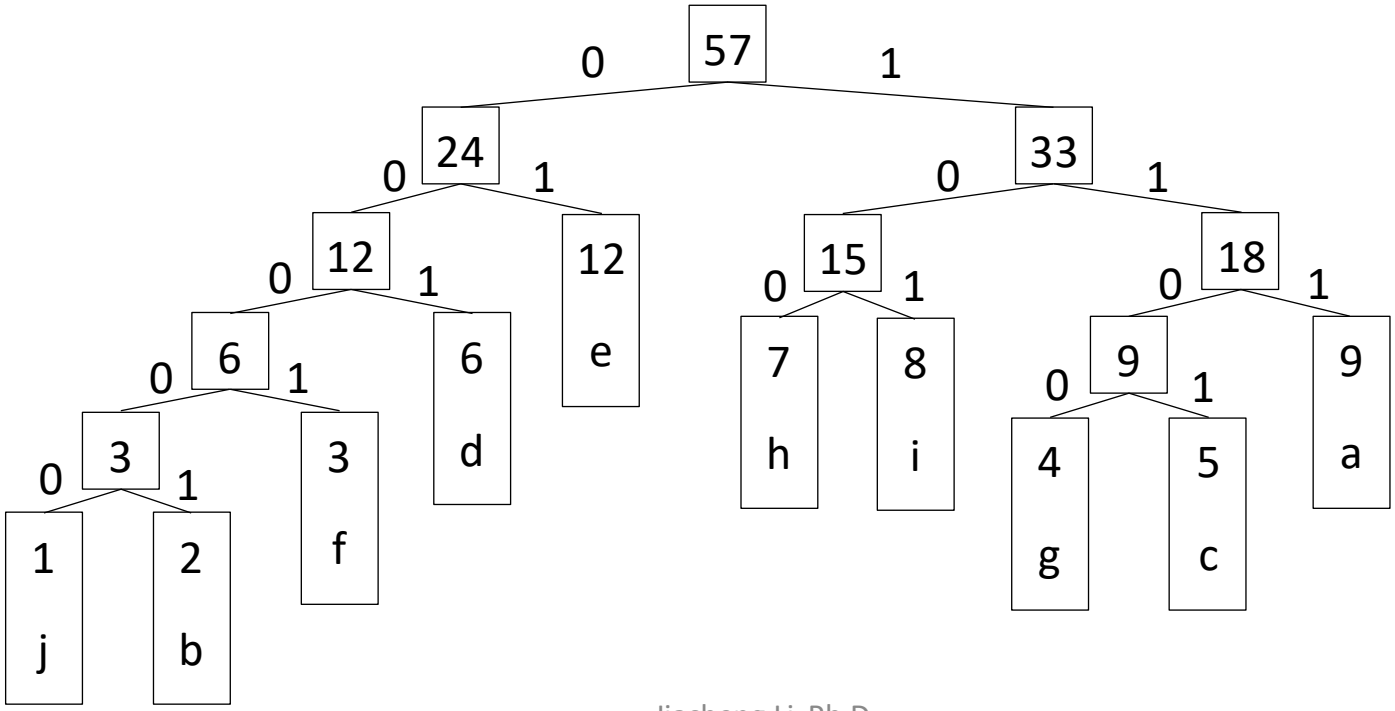
Greedy Huffman Algorithm (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	?						



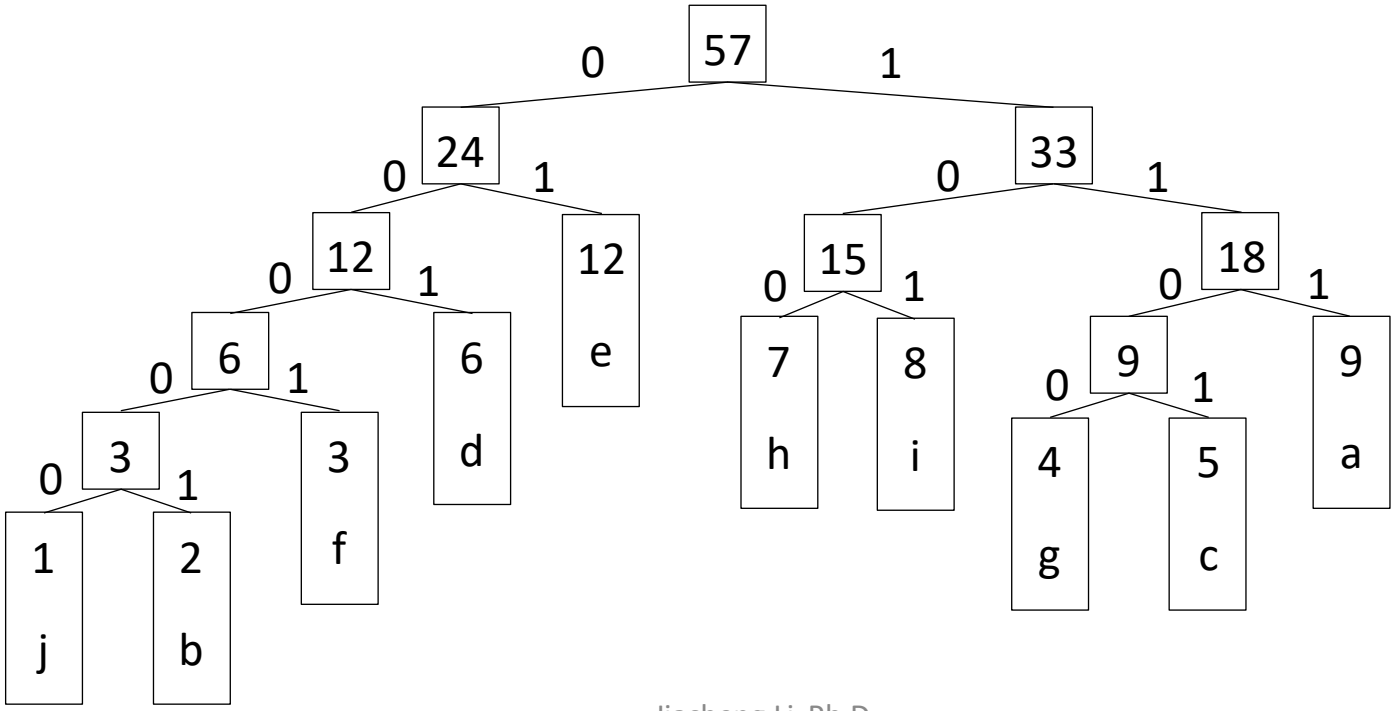
Greedy Huffman Algorithm (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	001	?					



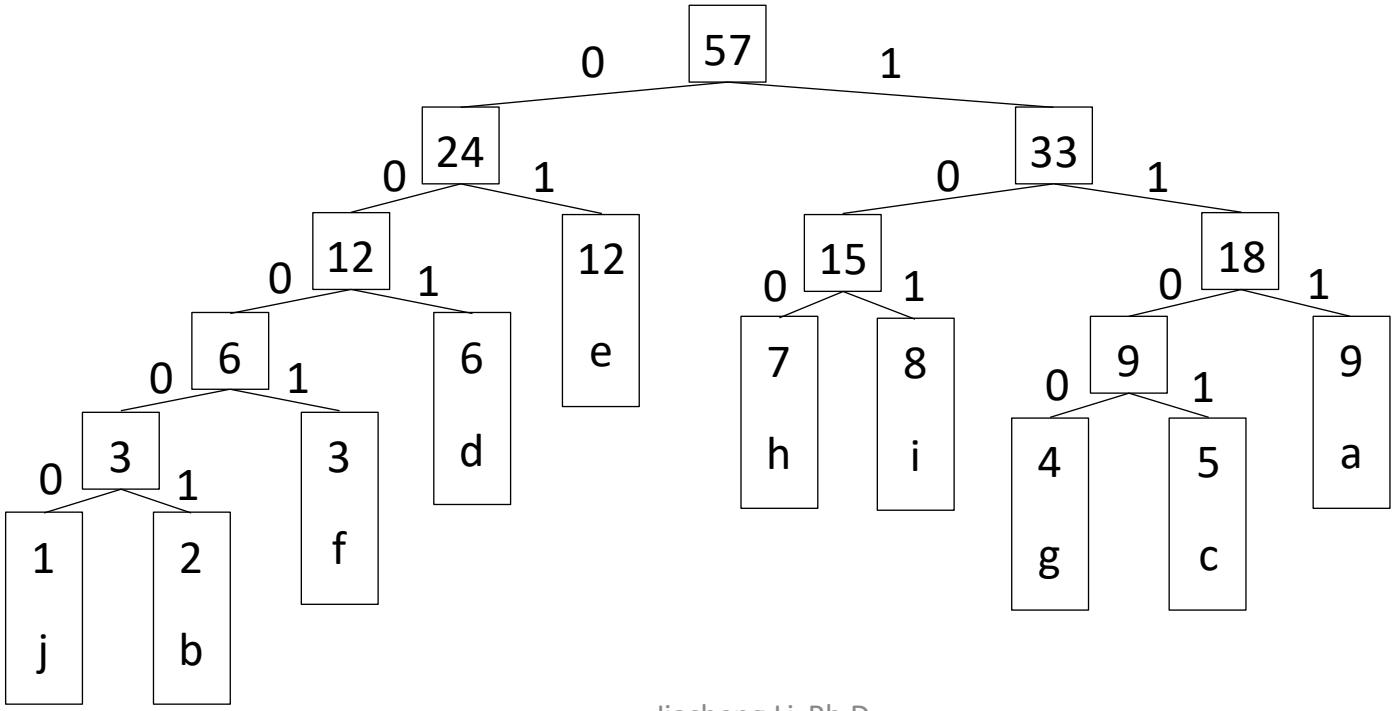
Greedy Huffman Algorithm (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	001	01	?				



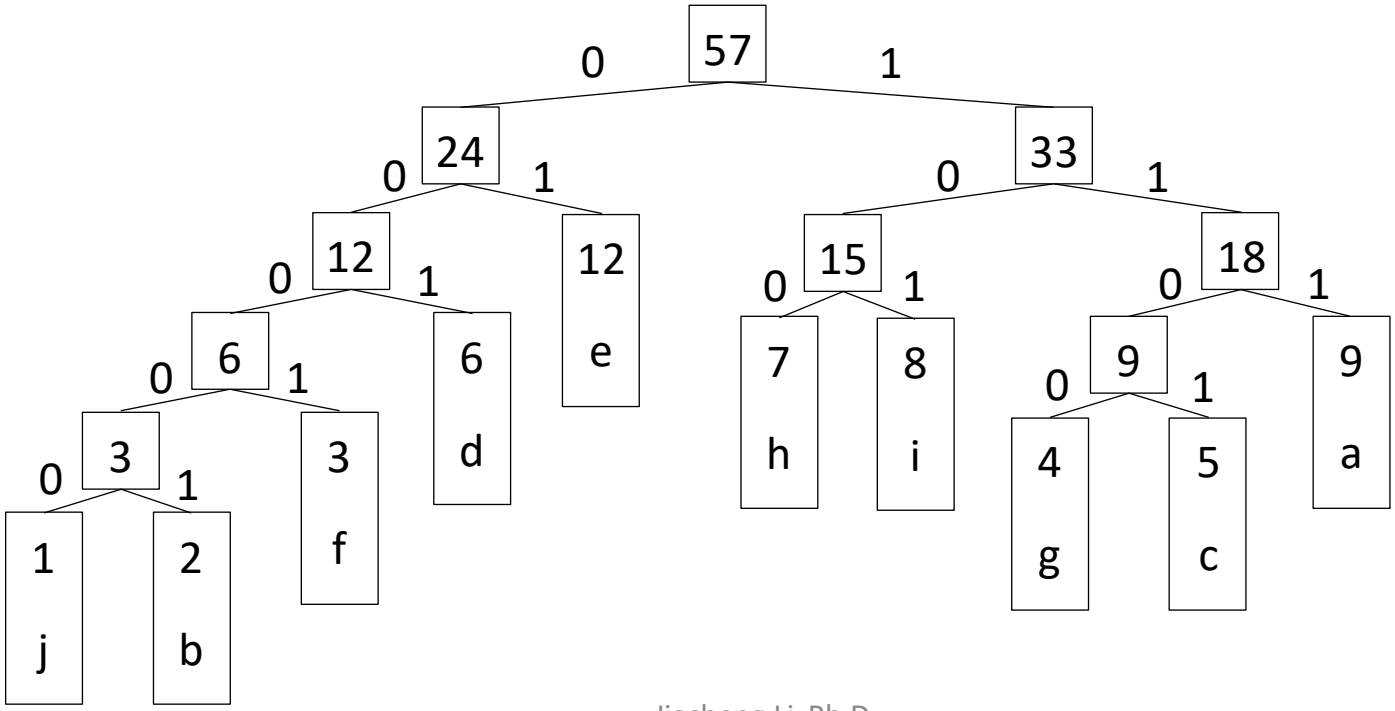
Greedy Huffman Algorithm (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	001	01	0001	?			



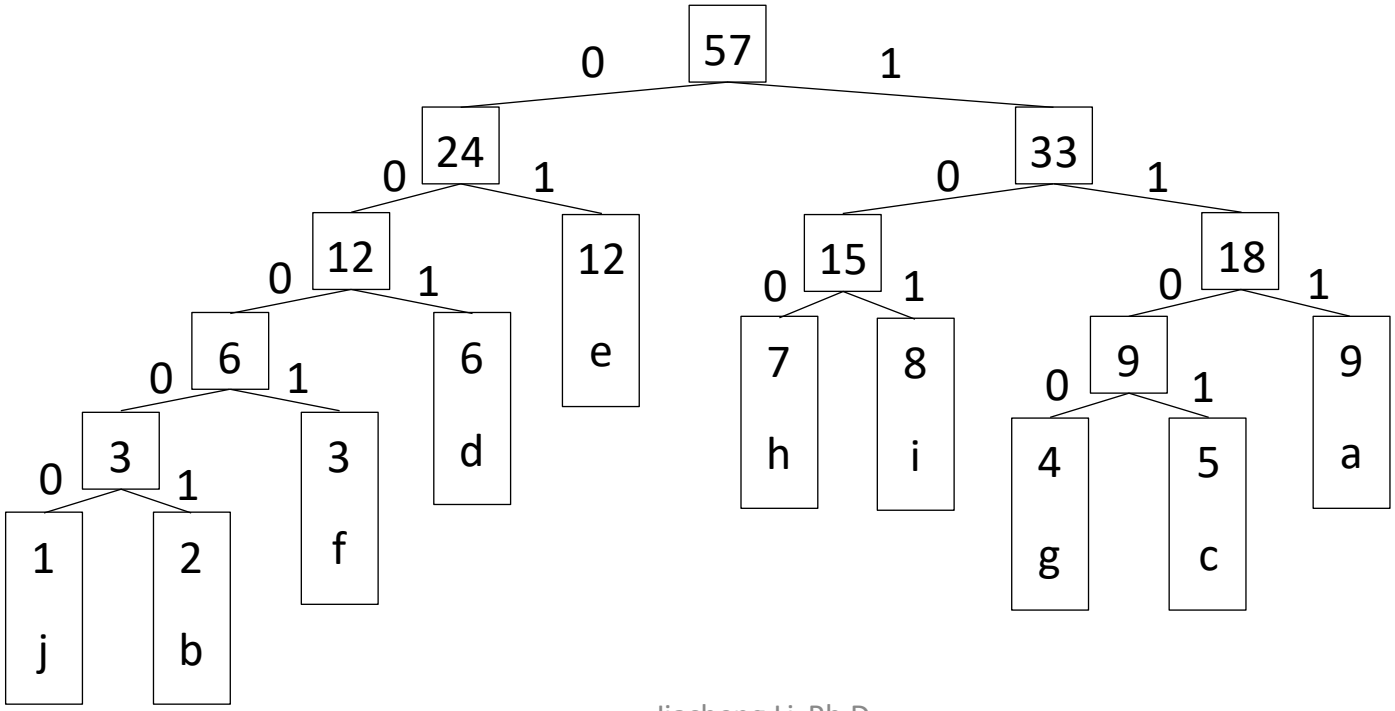
Greedy Huffman Algorithm (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	001	01	0001	1100	?		



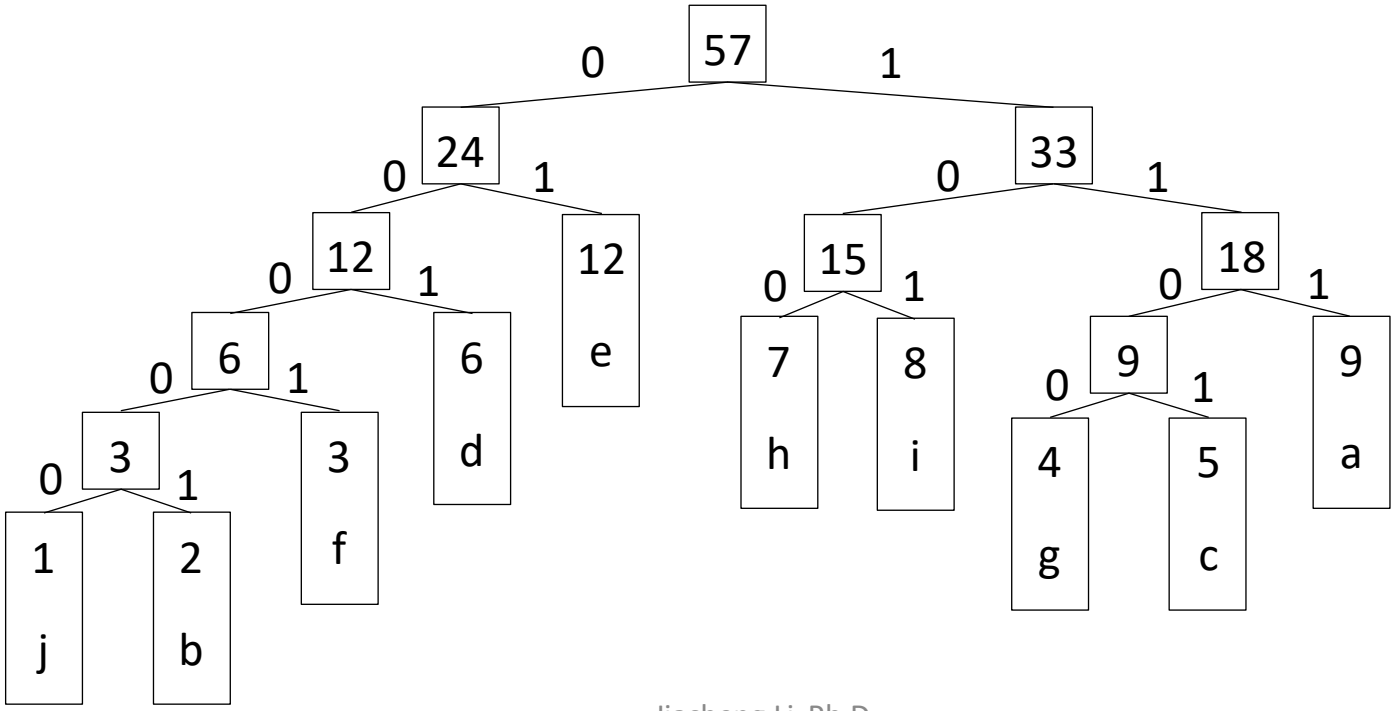
Greedy Huffman Algorithm (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	001	01	0001	1100	100	?	



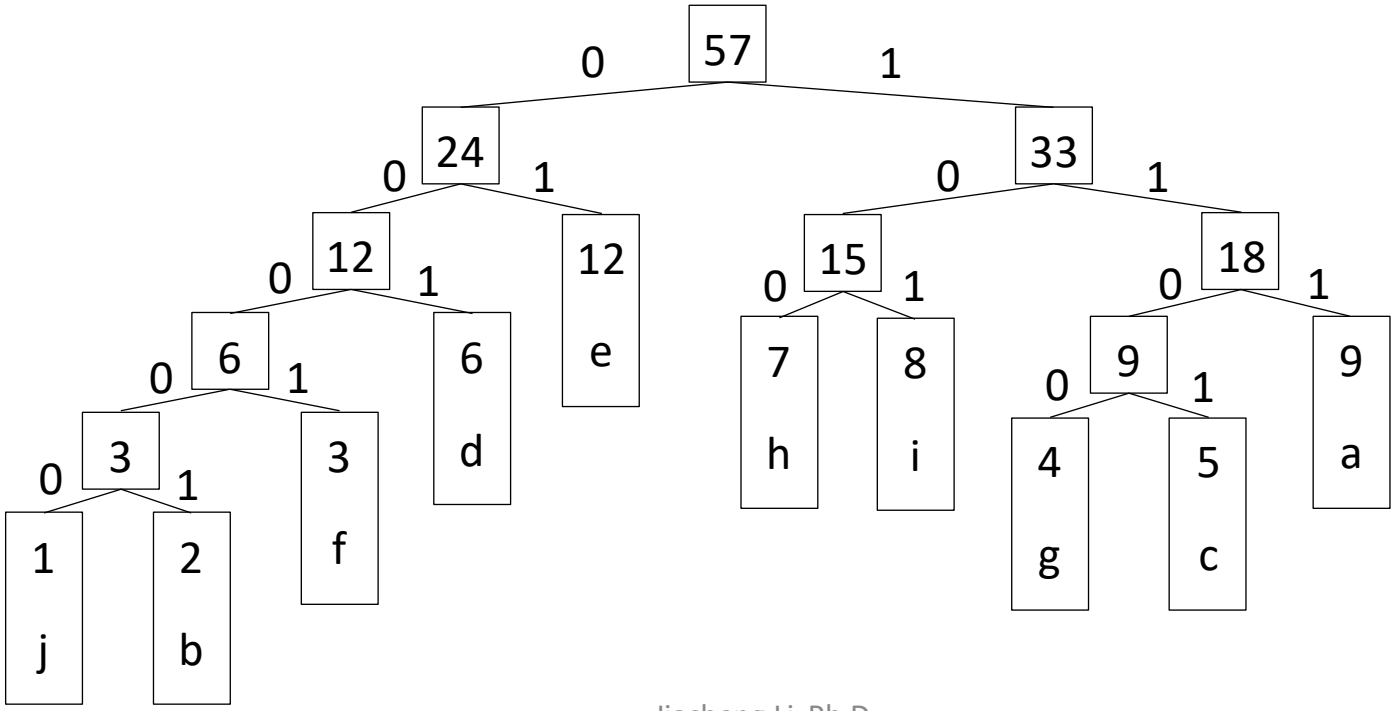
Greedy Huffman Algorithm (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	001	01	0001	1100	100	101	?



Greedy Huffman Algorithm (cont.)

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	001	01	0001	1100	100	101	00000



Encode

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	001	01	0001	1100	100	101	00000

- To encode a message, replace each character A[k] by its corresponding binary code C[k]
- Example
  - abigchefahead
  - a b i g c h e f a h e a d
  - 111 00001 101 1100 1101 100 01 0001 111 100 01 111 001

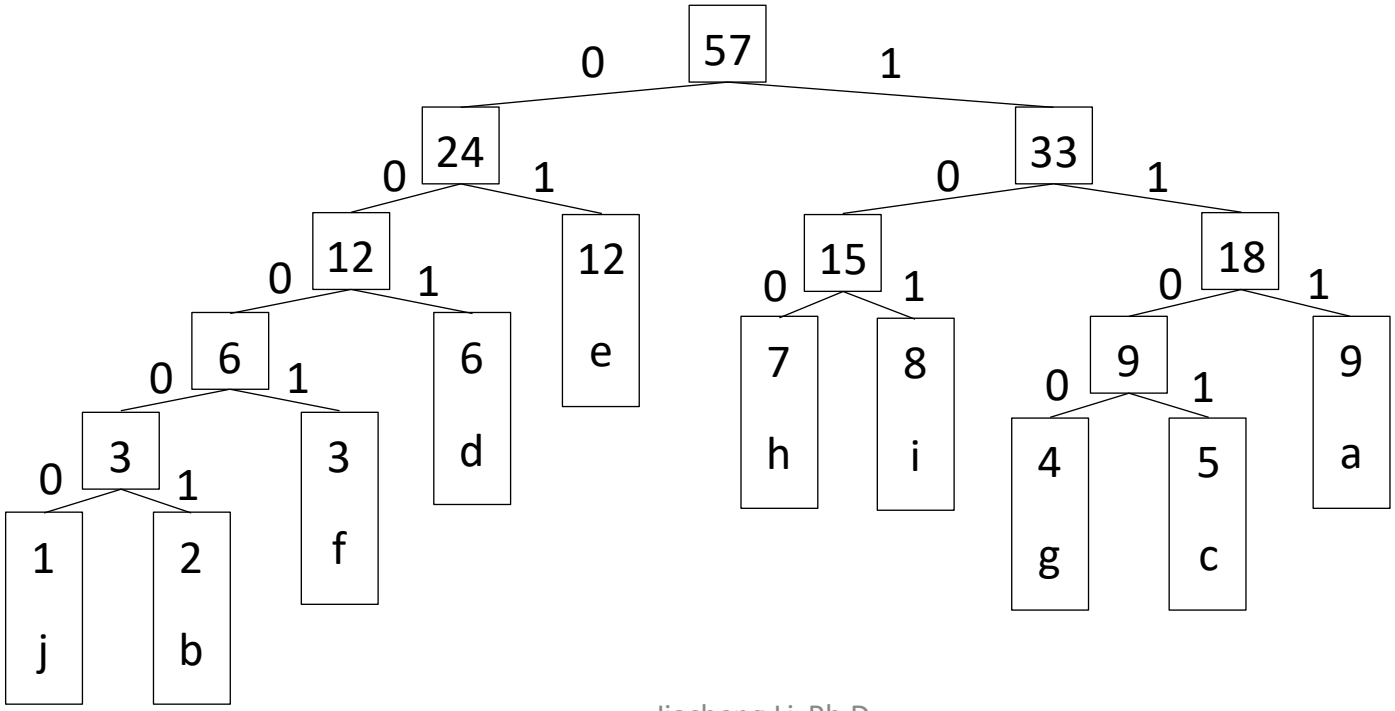
Decode

A	a	b	c	d	e	f	g	h	i	j
F	9	2	5	6	12	3	4	7	8	1
C	111	00001	1101	001	01	0001	1100	100	101	00000

- To decode a message, replace each binary code C[k] by its corresponding character A[k]
- Example
  - 1110000110111001101100010001111100011111001
  - How to replace?

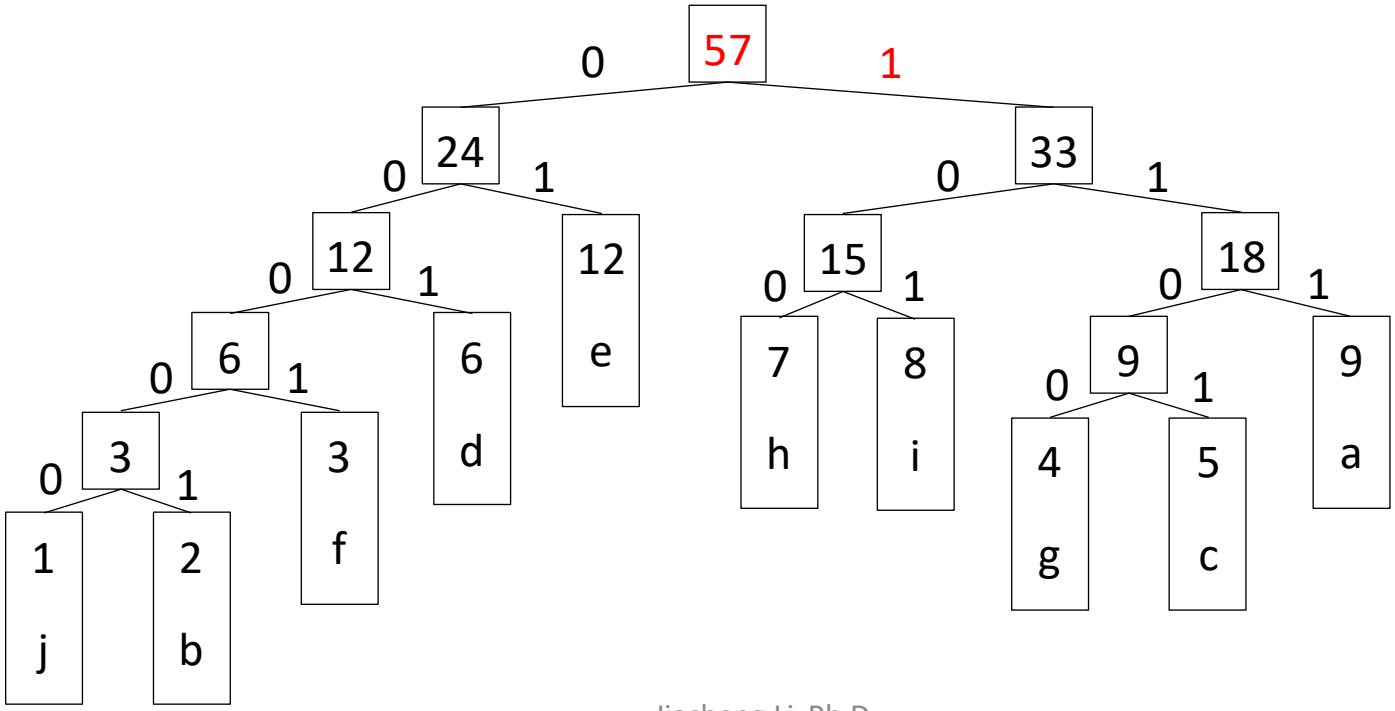
Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001



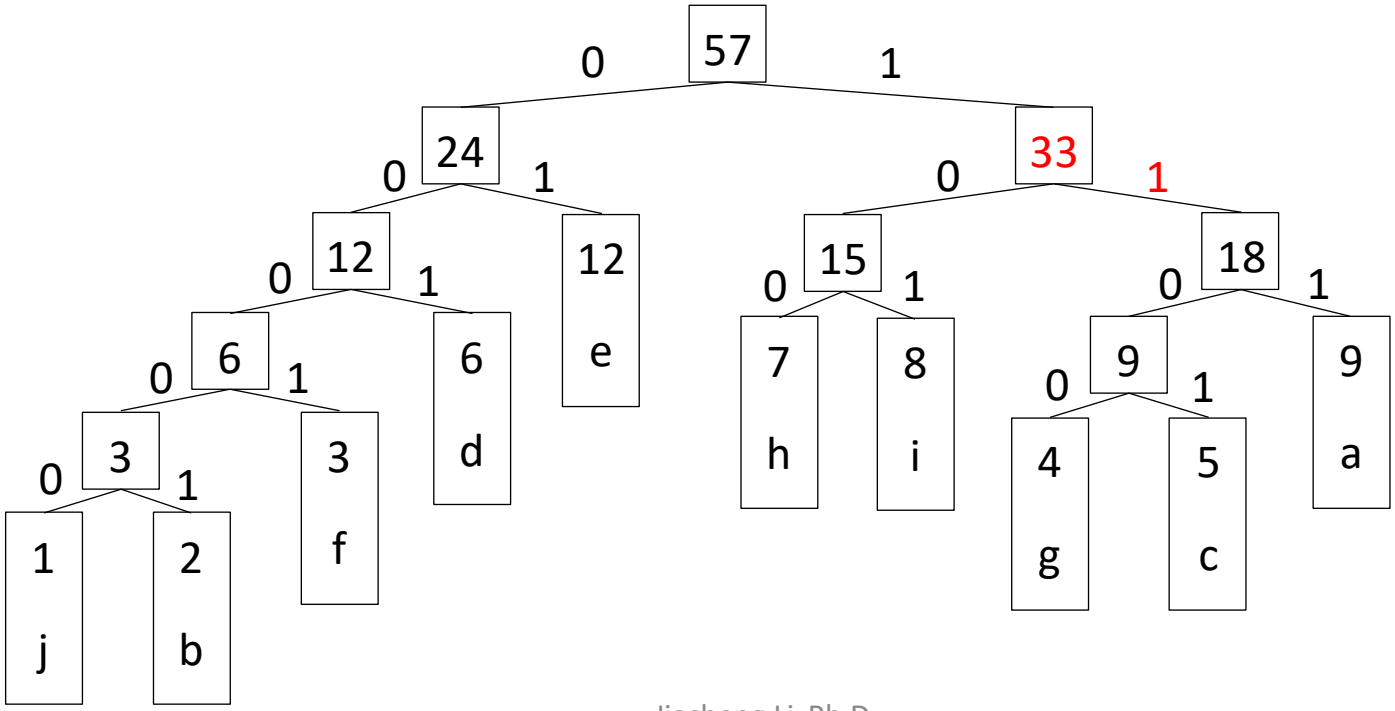
Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001



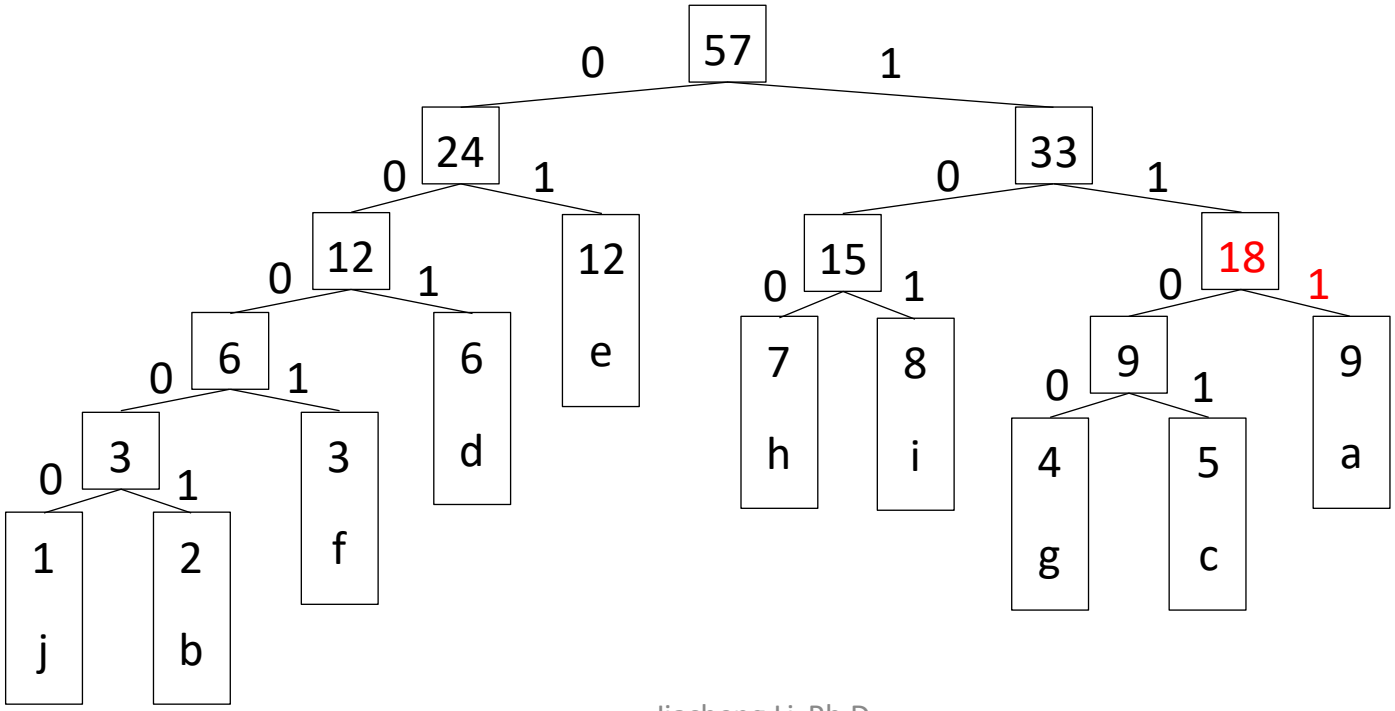
Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001



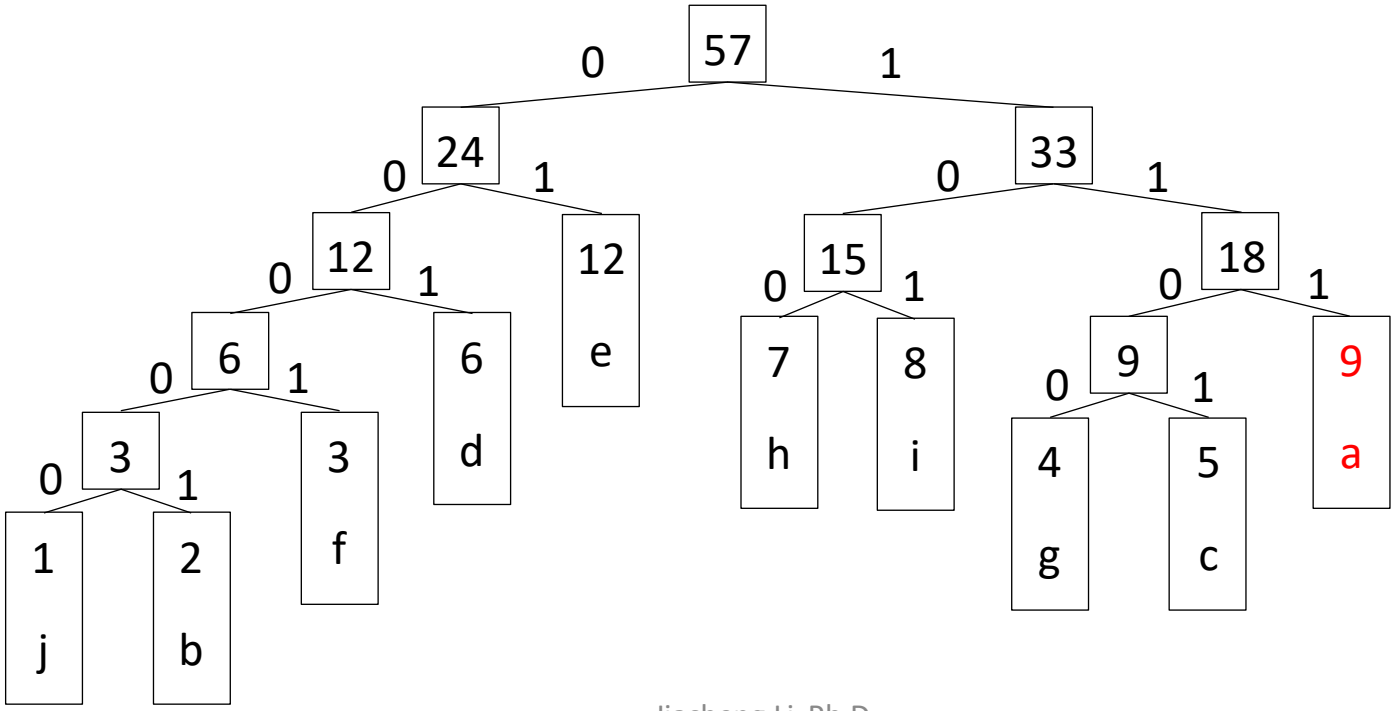
Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001



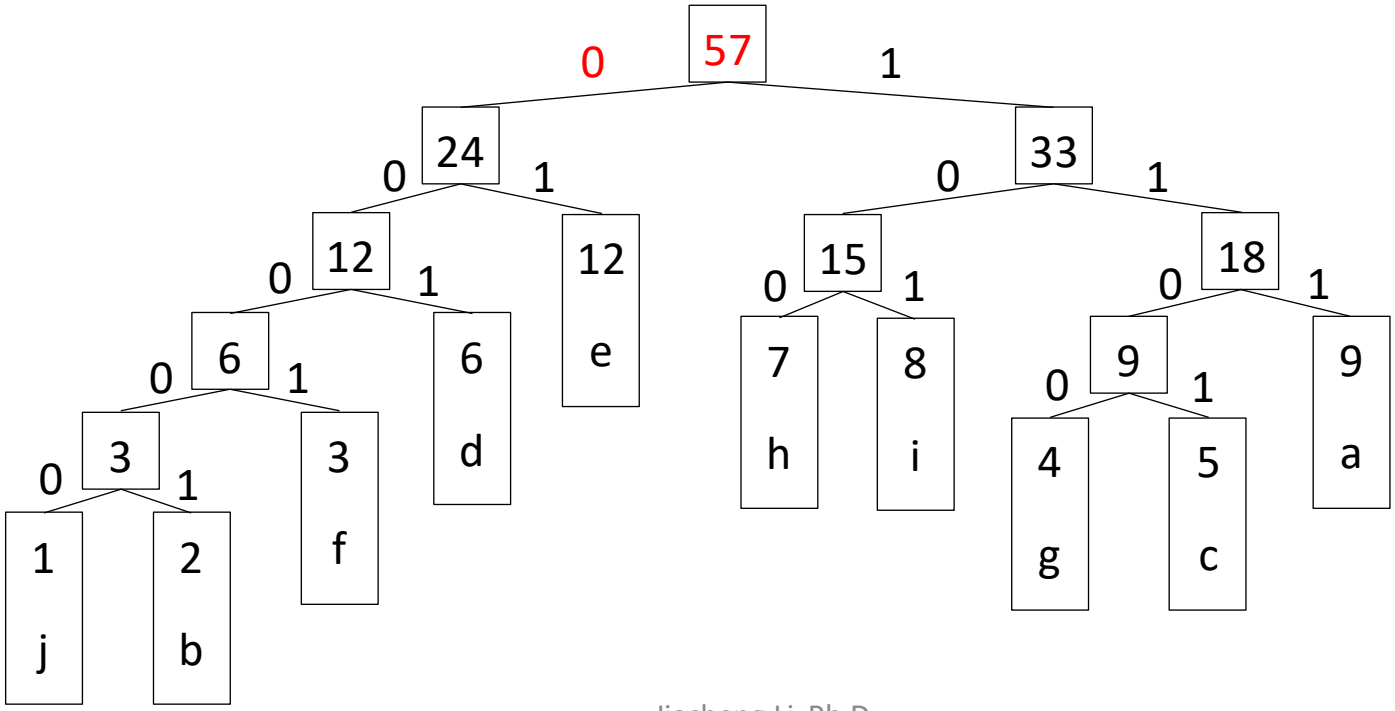
Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001
  - a



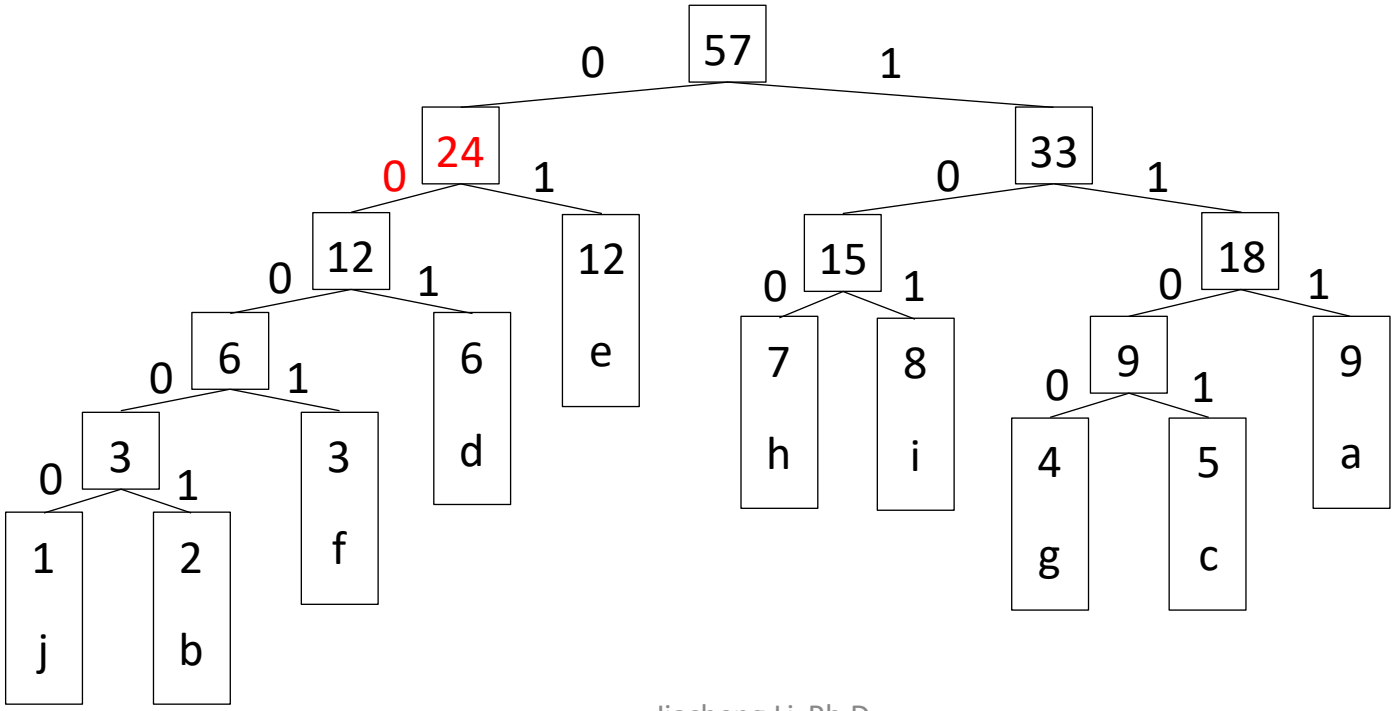
Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001
  - a



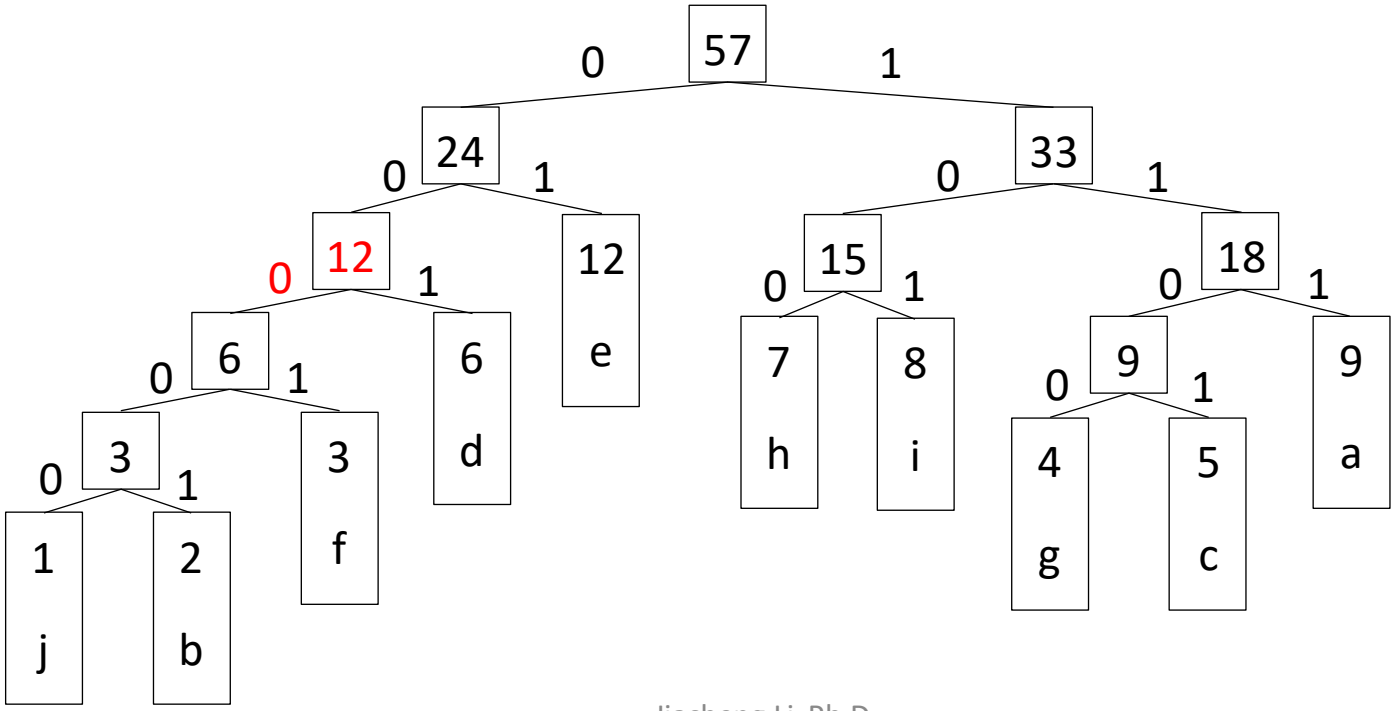
Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001
  - a



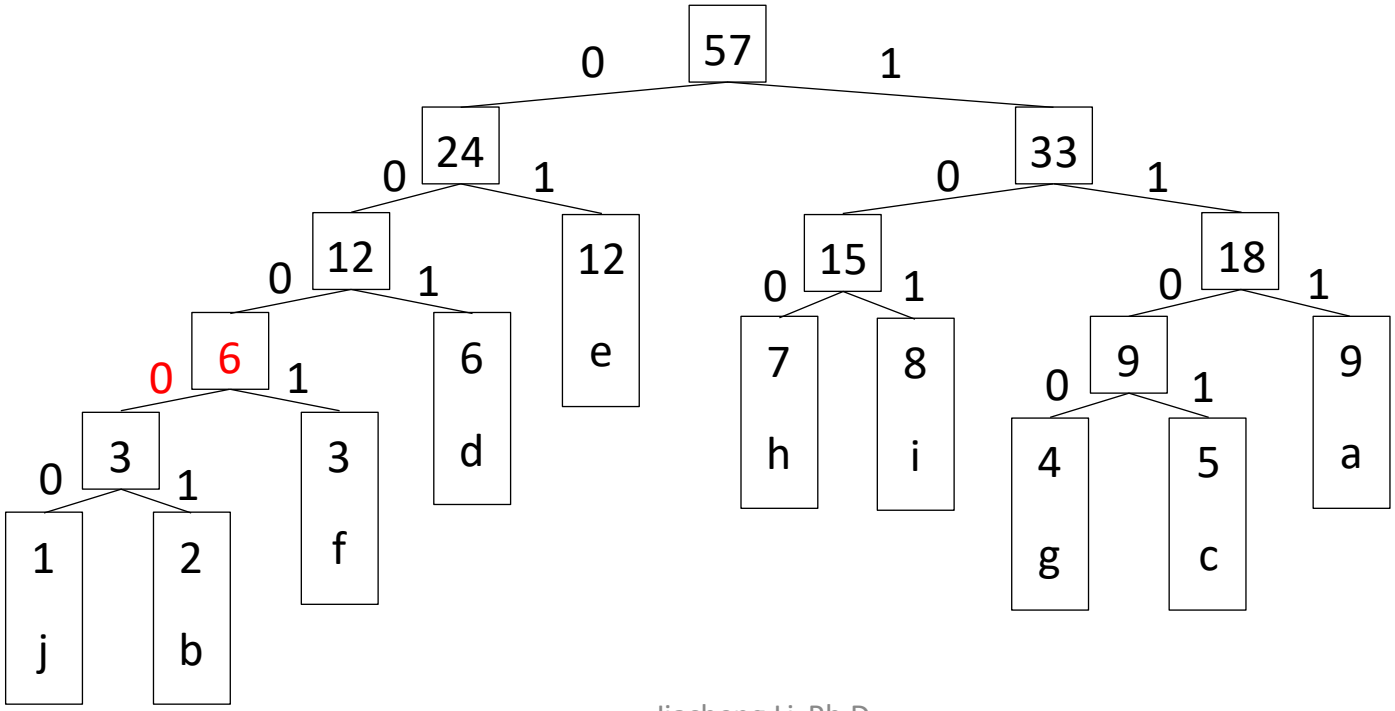
Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001
  - a



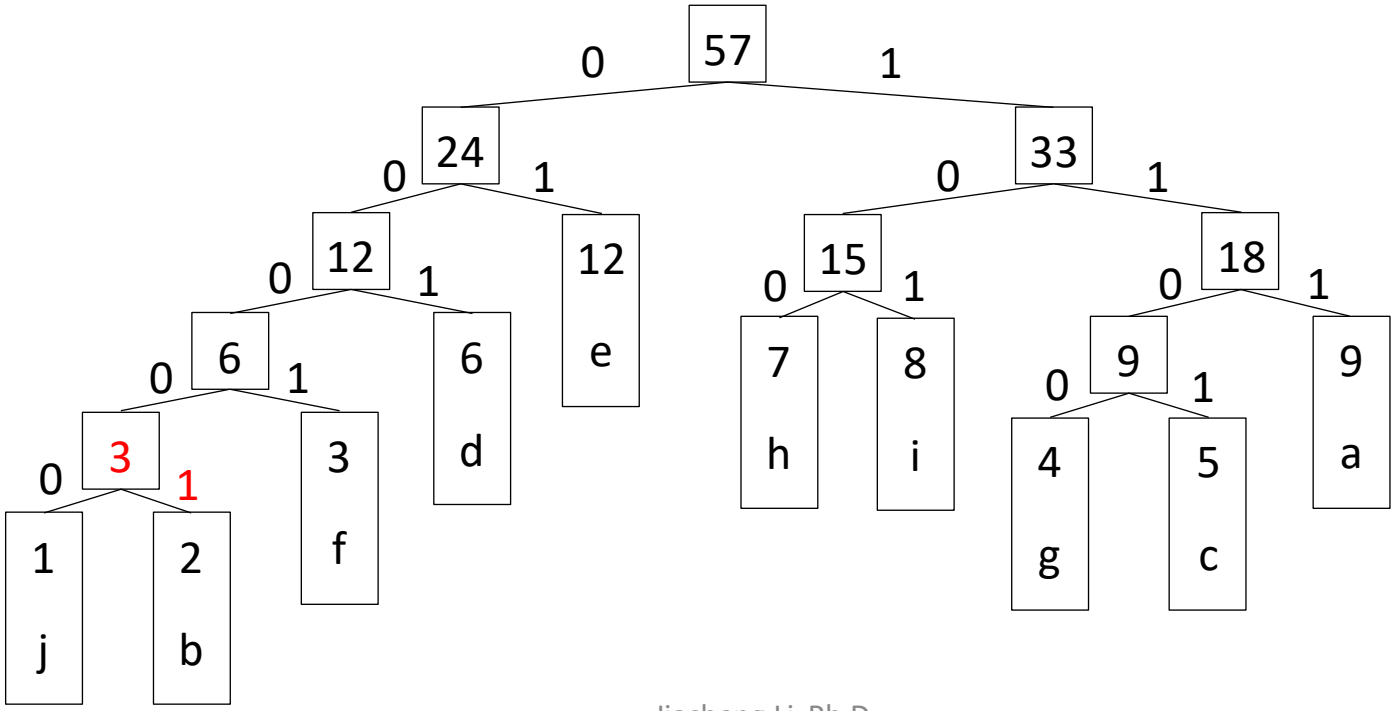
Decode (cont.)

- Example
  - 1110000110111001101100010001111100011110 01
  - a



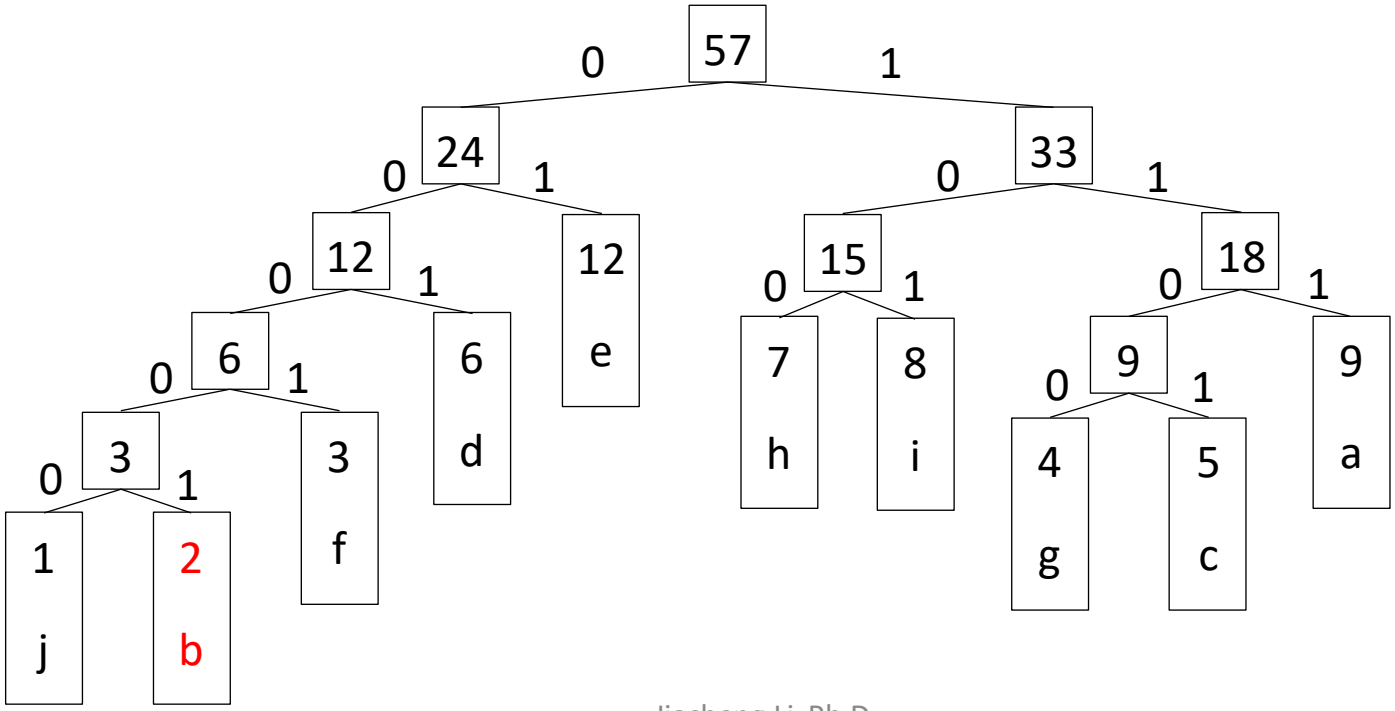
Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001
  - a



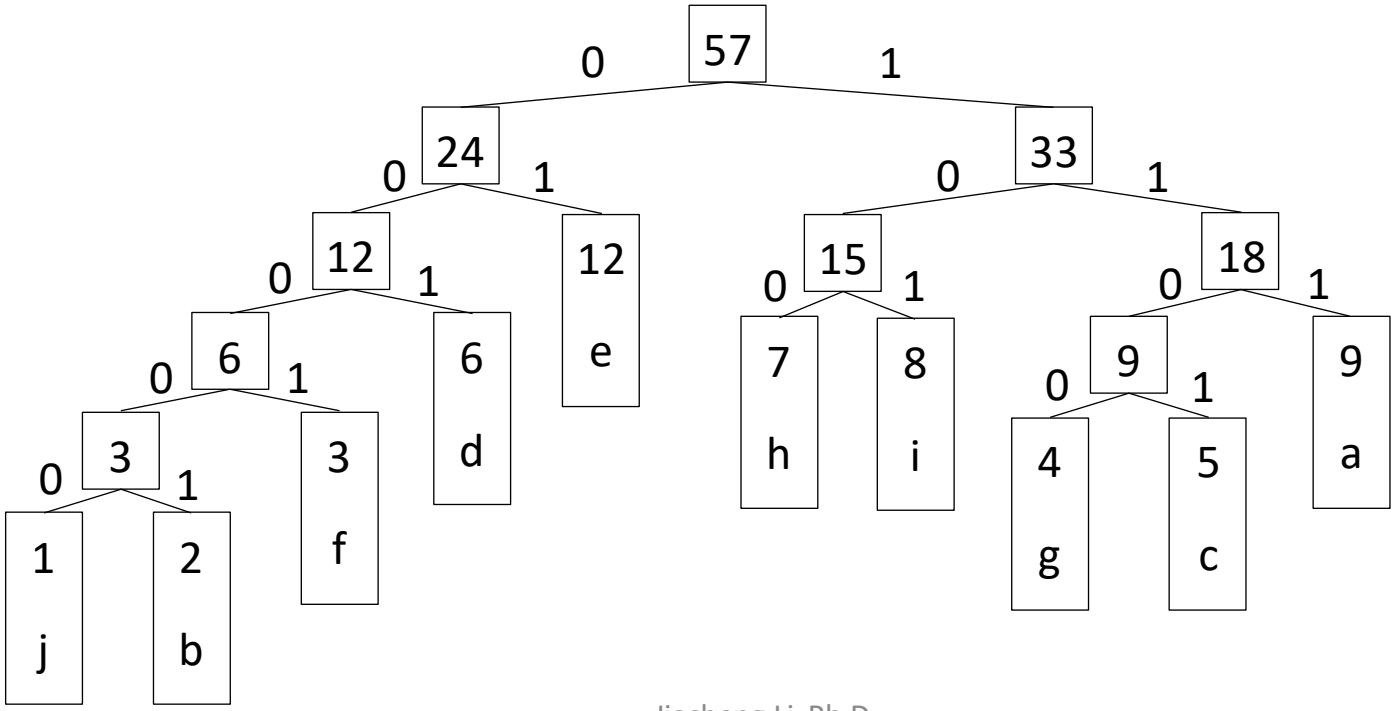
Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001
  - a b



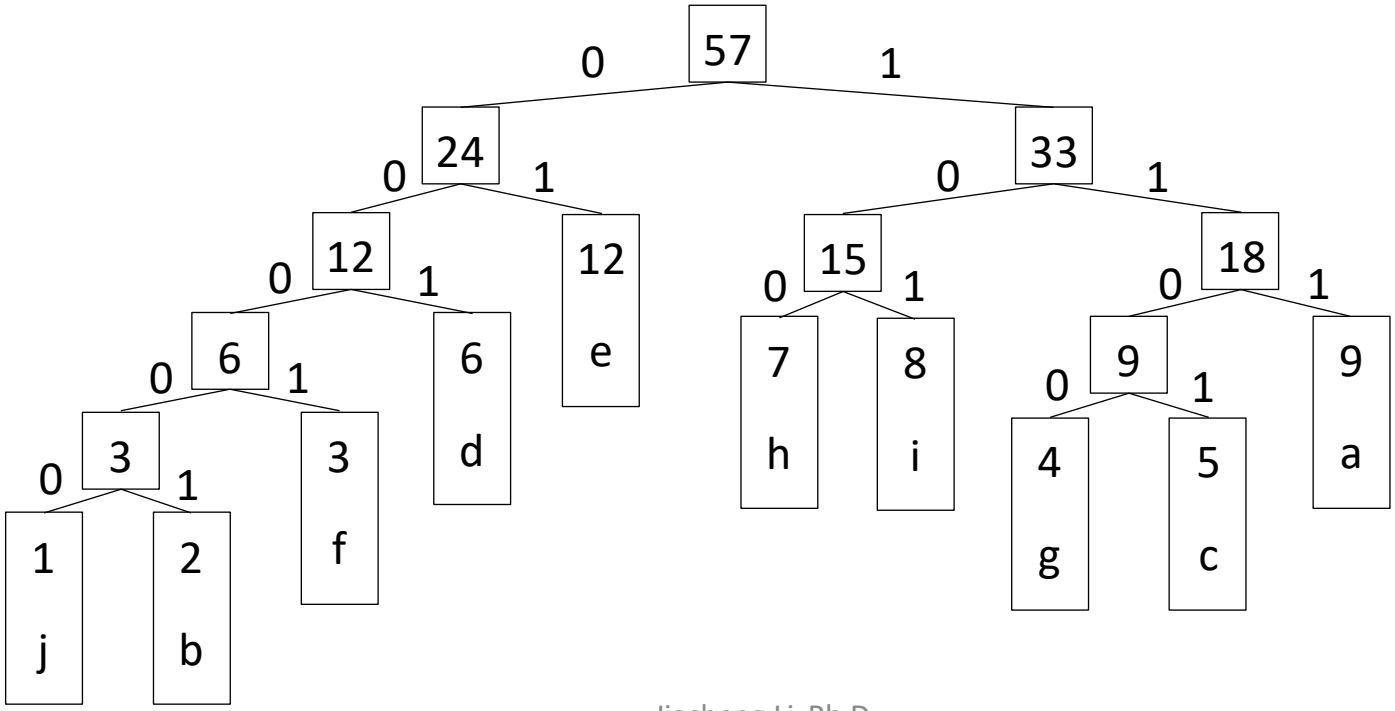
Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001
  - a b …



Decode (cont.)

- Example
  - 111000011011100110110001000111110001111001
  - abigchefahead



Questions?